

西南交通大学



云计算技术

课程设计报告

专业年级： 软件工程专业 2023 级

组员 1： 李欣纯（2023116052）

组员 2： 郑瑞璟（2023115271）

指导老师： 丛荣

提交日期： 2026 年 5 月 21 日

目录

1	课程设计概述	1
1.1	设计目标与任务要求	1
1.1.1	课程设计总体目标	1
1.1.2	第一部分云计算平台搭建要求	1
1.1.3	第二部分 Spark 大数据分析方向要求	2
1.2	小组分工与完成情况	2
1.2.1	成员分工比例	2
1.2.2	任务完成清单	3
1.2.3	代码仓库与报告材料说明	4
2	华为云实验环境	5
2.1	云服务与资源配置	5
2.1.1	SWR 镜像仓库配置	6
2.1.2	CCE 集群配置	6
2.1.3	OBS 数据存储配置	14
2.1.4	ELB 负载均衡配置	14
2.2	Kubernetes 集群信息	16
2.2.1	集群版本与节点规格	17
2.2.2	kubectl 连接与节点状态	18
2.2.3	命名空间与资源规划	19
3	第一部分：云计算平台搭建	20
3.1	任务 1：应用容器化	20
3.1.1	后端 Dockerfile 多阶段构建	20
3.1.2	后端依赖包修改与说明	21
3.1.3	前端首页学号姓名标识	21
3.1.4	Docker Compose 本地联调验证	21
3.1.5	SWR 镜像推送与镜像 Tag 验证	23
3.2	任务 2：CCE 集群搭建	25
3.2.1	CCE 集群创建过程	25
3.2.2	Worker 节点 Ready 状态验证	25
3.2.3	Kubernetes 版本要求验证	26
3.3	任务 3：应用部署	26
3.3.1	后端 Deployment 配置	27
3.3.2	Redis Deployment 配置	28
3.3.3	后端 LoadBalancer Service 配置	28

3.3.4	Redis ClusterIP Service 配置	28
3.3.5	ConfigMap 与 Secret 配置	29
3.3.6	Pod 运行状态与接口访问验证	29
3.4	任务 4: 持久化存储	31
3.4.1	Redis PVC 创建与绑定	31
3.4.2	Redis Deployment 挂载 PVC	31
3.4.3	Redis 测试数据写入	32
3.4.4	Redis Pod 删除与自动重建	32
3.4.5	重建后数据持久化验证	33
3.5	任务 5: ConfigMap Volume 挂载	33
3.5.1	Nginx 配置 ConfigMap 创建	33
3.5.2	前端 Deployment 挂载 ConfigMap	34
3.5.3	配置修改后的 Pod 内文件验证	35
3.5.4	Volume 挂载与 envFrom 适用场景对比	37
3.6	任务 6: HPA 弹性伸缩	37
3.6.1	metrics-server 可用性验证	38
3.6.2	后端 HPA 资源配置	38
3.6.3	压测方法与扩容过程截图	38
3.6.4	停止压测后的缩容过程截图	38
3.6.5	扩容延迟、冷却时间与降本价值分析	38
4	第二部分: Spark 大数据分析	39
4.1	A-0: Spark 环境部署	39
4.1.1	Spark Operator 安装	39
4.1.2	SparkApplication YAML 参数配置	40
4.1.3	WordCount 示例作业提交	40
4.1.4	Driver 与 Executor Pod 完成状态验证	42
4.2	A-1: 数据清洗	43
4.2.1	数据集选择与字段说明	44
4.2.2	DataFrame 加载与 Schema 输出	44
4.2.3	前 5 行数据预览	45
4.2.4	字段缺失值比例统计	45
4.2.5	缺失字段处理策略一	46
4.2.6	缺失字段处理策略二	46
4.2.7	清洗前后行数与统计信息对比	46
4.3	A-2: Spark SQL 统计分析	47
4.3.1	查询一: GROUP BY 聚合分析	48
4.3.2	查询二: ORDER BY Top-N 分析	48

4.3.3	查询三：时间维度趋势分析	49
4.3.4	查询四：JOIN 或窗口函数分析	51
4.3.5	统计结果截图与分析说明汇总	51
4.4	A-3：性能对比与 Amdahl 分析	52
4.4.1	对比查询选择与实验设计	52
4.4.2	Pandas 单机执行时间	52
4.4.3	PySpark 单 Executor 执行时间	53
4.4.4	PySpark 双 Executor 执行时间	54
4.4.5	性能对比图绘制	56
4.4.6	Amdahl 定律加速比分析	56
4.4.7	非线性加速原因讨论	57
5	问题记录与解决方案	57
5.1	第一部分问题记录	58
5.1.1	镜像构建与推送问题	58
5.1.2	CCE 部署与网络访问问题	58
5.1.3	PVC 与 HPA 验证问题	59
5.2	第二部分问题记录	59
5.2.1	Spark 作业提交问题	59
5.2.2	OBS 或本地数据读取问题	60
5.2.3	性能测试误差问题	60
6	总结与收获	60
6.1	云计算平台搭建总结	61
6.1.1	容器化与 Kubernetes 部署理解	61
6.1.2	配置分离、持久化与弹性伸缩理解	61
6.2	Spark 大数据分析总结	62
6.2.1	分布式数据处理流程理解	62
6.2.2	性能对比与 Amdahl 定律理解	62
6.3	课程设计反思	63
6.3.1	主要挑战	63
6.3.2	改进方向	63
A	附录：核心配置与代码	63
A.1	Docker 相关文件	63
A.1.1	后端 Dockerfile	63
A.1.2	前端 Dockerfile	64
A.1.3	docker-compose.yml	64
A.2	Kubernetes YAML 文件	64

A.2.1	Deployment 与 Service 配置	64
A.2.2	ConfigMap 与 Secret 配置	64
A.2.3	PVC 与 HPA 配置	64
A.3	Spark 作业代码	64
A.3.1	wordcount.py	64
A.3.2	analysis.py	64
A.3.3	性能测试脚本	64
B	附录：实验截图索引	64
B.1	第一部分截图索引	65
B.1.1	任务 1 至任务 3 截图	65
B.1.2	任务 4 至任务 6 截图	65
B.2	第二部分截图索引	65
B.2.1	A-0 与 A-1 截图	65
B.2.2	A-2 与 A-3 截图	65

第一章 课程设计概述

1.1 设计目标与任务要求

本课程设计围绕云计算平台搭建与并行编程实战两条主线展开，目标是在真实云平台环境中完成从应用容器化、镜像仓库管理、Kubernetes 集群部署，到分布式数据分析与性能评价的完整实践。与单纯的本地实验不同，本项目要求将课堂中涉及的容器、服务发现、负载均衡、持久化存储、配置分离、弹性伸缩和分布式计算等概念，落实到华为云 CCE、SWR、ELB、OBS 等具体服务之上。因此，本报告不只记录操作过程，也重点说明各组件之间的关系、关键配置的作用以及实验验证的依据。

根据任务书要求，课程设计总成绩由云计算平台搭建、并行编程实战和报告质量三部分构成。其中，第一部分要求在华为云 CCE 上部署 Flask 后端 API、Redis 数据库和 Nginx 前端页面，并完成对外暴露、数据持久化、ConfigMap Volume 挂载和 HPA 弹性伸缩验证；第二部分选择方向 A，即 Spark 大数据分析，围绕豆瓣电影数据集完成 Spark Operator 环境部署、数据清洗、Spark SQL 统计分析以及 Pandas 与 PySpark 的性能对比；报告部分则要求结构完整、截图清晰、分析充分，并附上核心代码和配置文件。

1.1.1 课程设计总体目标

本项目的总体目标可以概括为“构建一套可运行、可验证、可分析的云原生课程设计系统”。在平台层面，项目以容器化 Web 应用为载体，将后端服务、缓存数据库和前端静态服务分别构建为容器镜像，并通过 Kubernetes Deployment、Service、ConfigMap、Secret、PVC 和 HPA 等资源完成部署与治理。在数据计算层面，项目选择 Spark 大数据分析方向，利用 PySpark 对电影数据进行清洗、聚合统计、趋势分析和性能测试，体现云平台对并行计算任务的支撑能力。

从能力训练角度看，本课程设计主要覆盖以下目标：第一，掌握 Docker 镜像构建、Docker Compose 本地联调和 SWR 镜像推送流程；第二，掌握 CCE 集群创建、kubectl 连接、Pod 与 Service 状态检查等 Kubernetes 基础操作；第三，理解 ConfigMap、Secret、PVC、LoadBalancer、HPA 等资源在云原生应用中的作用；第四，掌握 Spark DataFrame、Spark SQL 和分布式执行模式的基本使用；第五，能够结合截图、日志、性能数据和理论模型对实验结果进行解释。

1.1.2 第一部分云计算平台搭建要求

第一部分的核心任务是完成一套基于华为云 CCE 的云计算平台搭建，满分 50 分。该部分从应用容器化开始，先对 Flask 后端和 Nginx 前端分别编写 Dockerfile，并通过 Docker Compose 在本地验证后端与 Redis 的通信；随后将镜像推送至华为云 SWR 镜像仓库，为云端部署提供镜像来源。在云平台侧，需要创建满足版本要求的 CCE 集群，

配置不少于两个 Worker 节点，并通过 kubectl 验证节点均处于 Ready 状态。

应用部署阶段要求将后端、Redis 和前端部署到 CCE 集群中。后端 Deployment 需要设置两个副本，并配置资源请求与限制；Redis 作为内部数据库服务，通过 ClusterIP Service 在集群内提供访问；后端 Service 使用 LoadBalancer 类型并绑定华为云 ELB，从而对外暴露 ‘/api/ping’ 接口。配置管理方面，后端通过 ConfigMap 获取 Redis 地址和端口，通过 Secret 获取敏感密码；存储方面，Redis 通过 PVC 挂载持久化目录，验证 Pod 删除重建后数据不丢失；前端 Nginx 配置则采用 ConfigMap Volume 方式挂载，以体现配置与镜像解耦。最后，后端服务还需要配置 HPA，并通过压测观察副本扩缩容过程。

1.1.3 第二部分 Spark 大数据分析方向要求

第二部分选择任务书中的方向 A：Spark 大数据分析，满分 40 分。该方向要求在 Kubernetes 环境中通过 Spark Operator 运行 Spark 作业，并以 PySpark 为主要工具完成数据清洗、统计查询和性能分析。环境部署部分需要安装 Spark Operator，修改 SparkApplication 模板中的镜像、Executor 数量和内存配置，并运行 WordCount 示例作业，验证 Driver Pod 和 Executor Pod 能够正常创建和完成。

数据处理部分以豆瓣电影评分数据集为分析对象。报告需要给出数据加载过程、DataFrame Schema、前 5 行样例、字段缺失值比例，并至少对两个存在缺失值的字段采用不同清洗策略。统计分析部分要求完成不少于 4 个查询，且必须覆盖 GROUP BY 聚合、ORDER BY Top-N、按时间维度的趋势分析，以及 JOIN 或窗口函数。性能分析部分需要选取一个查询，分别使用 Pandas 单机、PySpark 单 Executor 和 PySpark 双 Executor 运行，记录执行时间并绘制对比图，再结合 Amdahl 定律分析加速比未达到线性增长的原因。

1.2 小组分工与完成情况

本课程设计由李欣纯和郑瑞璟共同完成。整体分工遵循“云平台实施”和“应用开发、数据分析及报告整合”并行推进的原则：一方面，由一名成员集中负责 CCE、Kubernetes、PVC、ConfigMap Volume、HPA、Spark Operator 等云端资源，降低多人交叉修改集群配置造成的不一致风险；另一方面，由另一名成员负责应用容器化、镜像构建、Spark 数据清洗、SQL 分析、性能对比和报告主笔，从而保证代码、实验数据和文字材料能够统一整理。两位成员最终工作量按项目实施难度、排错成本、截图整理和报告撰写综合计算，约为 1:1。

1.2.1 成员分工比例

小组分工如表 1 所示。云平台相关任务依赖真实华为云资源，涉及网络、镜像拉取、Service 绑定、PVC 和 HPA 等运行时问题，环境调试成本较高；Spark 数据分析

与报告整理则涉及代码实现、统计结果解释、性能图表绘制和文档排版，分析与表达工作量较大。两部分互相补充，使项目能够在功能实现和报告质量之间保持平衡。

表 1: 小组成员分工概览

成员	主要职责	具体工作内容	工作量
李欣纯	应用开发、数据分析与报告整合	负责 Dockerfile 修改、本地联调、镜像构建与推送、Spark 数据清洗、Spark SQL 统计分析、性能对比与 Amdahl 分析，并承担报告结构设计、文字撰写、图表整理和最终排版。	约 50%
郑瑞璟	云 平 台 部 署、Kuber- netes 运维与云端验证	负责 CCE 集群创建、kubectl 配置、Kubernetes 应用部署、Redis 持久化、ConfigMap Volume 挂载、HPA 验证、Spark Operator 环境部署材料准备及云端截图整理。	约 50%

1.2.2 任务完成清单

根据任务书评分标准,本报告后续章节将按照任务编号逐项展开。第一部分围绕 T1 至 T6 组织,分别对应应用容器化、CCE 集群搭建、应用部署、持久化存储、ConfigMap Volume 挂载和 HPA 弹性伸缩;第二部分围绕 Spark 方向 A-0 至 A-3 组织,分别对应 Spark 环境部署、数据清洗、SQL 统计分析和性能对比。每一项任务均尽量采用“操作目标—关键配置—验证截图—结果分析”的写法,避免只罗列命令而缺少原理解释。

为了直观展示项目当前已经形成的实验材料,图 1 至图 3 选取了本地联调、CCE 节点状态和云端接口访问三类代表性截图。这些截图分别对应从本地容器运行到云端集群部署再到公网服务验证的关键路径,能够体现项目不是停留在静态配置层面,而是完成了实际运行和结果验证。

```

PS C:\WINDOWS\system32> cd D:\cloud-course-project
PS D:\cloud-course-project> docker compose up -d --build
time="2026-06-09T16:04:58+08:00" level=warning msg="D:\\cloud-course-project\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
#1 [internal] load local bake definitions
#1 reading from stdin 549B done
#1 DONE 0.0s
#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 601B done
#2 DONE 0.0s
#3 [internal] load metadata for docker.io/library/python:3.11-slim
#3 DONE 0.0s
#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s
#5 [internal] load build context
#5 transferring context: 93B done
#5 DONE 0.0s
#6 [builder 1/4] FROM docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9fc0e6e38295747e49ac0
#6 resolve docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9fc0e6e38295747e49ac0 0.0s done
#6 DONE 0.0s
#7 [builder 3/4] COPY requirements.txt .
#7 CACHED
#8 [stage-1 2/4] WORKDIR /app
#8 CACHED
#9 [builder 2/4] WORKDIR /build
#9 CACHED
#10 [builder 4/4] RUN pip install --no-cache-dir -r requirements.txt --target /build/packages
#10 CACHED
#11 [stage-1 3/4] COPY --from=builder /build/packages /app/packages
#11 CACHED
#12 [stage-1 4/4] COPY . .
#12 CACHED
#13 exporting to image
#13 exporting layers done
#13 exporting manifest sha256:6d4674a6a81f741ae27c9add732c9ff406e9e9191b6bf523fba19545499f80f4 done
#13 exporting config sha256:1685f82b2a70ccc2bf1225acec4f13c065fe85540bbb47e9d0e5065f8b856655 done
#13 exporting attestation manifest sha256:31d790267626b2950753f72310a45f802e458963ad1dab12c76405f043ad2045 0.0s done
#13 exporting manifest list sha256:e670744770787b39fcb5a0cd7c5e490f1cb80030c58483098e82ebcb50bc1ee0
#13 exporting manifest list sha256:e670744770787b39fcb5a0cd7c5e490f1cb80030c58483098e82ebcb50bc1ee0 0.0s done
#13 naming to docker.io/library/cloud-course-project-backend:latest done
#13 unpacking to docker.io/library/cloud-course-project-backend:latest done
#13 DONE 0.1s

```

图 1: Docker Compose 本地联调启动成功截图

```

终端0 x
user@e8pgt9kmw5c5ycz-machine:~$ kubectl get nodes -o wide
NAME                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION
CONTAINER-RUNTIME
192.168.0.215       Ready    <none>    75m   v1.34.6-r0-34.0.23   192.168.0.215   <none>        Ubuntu 22.04.5 LTS   5.15.0-173-generic
containerd://1.7.29-19-g707eaa844
192.168.0.237       Ready    <none>    75m   v1.34.6-r0-34.0.23   192.168.0.237   <none>        Ubuntu 22.04.5 LTS   5.15.0-173-generic
containerd://1.7.29-19-g707eaa844
user@e8pgt9kmw5c5ycz-machine:~$

```

图 2: CCE 集群两个 Worker 节点 Ready 验证截图

```

user@dhogkgjdvsjy2ai-machine:~$ curl http://1.92.151.224/api/ping
{"redis":"connected","status":"ok"}
user@dhogkgjdvsjy2ai-machine:~$

```

图 3: 通过 ELB 公网入口访问后端接口的验证截图

1.2.3 代码仓库与报告材料说明

本项目的代码与实验材料统一保存在课程设计仓库中。仓库根目录包含 ‘backend’、‘frontend’、‘k8s’、‘spark’、‘figures’、‘figures_A’ 等目录，分别用于保存后端应用、前端静态页面、Kubernetes 资源文件、Spark 分析脚本和实验截图。后端部分以 Flask 提供 ‘/api/ping’ 接口，并通过 Redis 验证服务间通信；前端部分基于 Nginx 提供静态页面和反向代理配置；‘k8s’ 目录保存 Deployment、Service、ConfigMap、Secret、PVC、HPA 等 YAML 文件；‘spark’ 目录保存 WordCount 示例、SparkApplication 模板和豆瓣电影数据分析脚本。

报告截图主要来自 ‘figures’ 和 ‘figures_A’ 目录。‘figures/T1’ 保存应用容器化、本地联调和 SWR 镜像推送相关截图；‘figures_A/T2’ 至 ‘figures_A/T5’ 保存 CCE 集群、

应用部署、Redis 持久化和 ConfigMap Volume 挂载相关截图。后续章节将根据任务书验收要求插入截图，并在正文中说明截图反映的关键信息，例如 Pod 状态、Service 类型、ELB 公网 IP、PVC Bound 状态、Redis 数据持久化结果以及接口返回值等。通过这种组织方式，报告正文、代码文件和截图证据之间形成一一对应关系，便于检查和答辩说明。

第二章 华为云实验环境

本章对课程设计使用的华为云实验环境进行说明。按照任务书要求，本项目主要使用华为云 SWR、CCE、OBS 和 ELB 四类服务：SWR 用于保存后端和前端容器镜像；CCE 用于提供托管 Kubernetes 集群；OBS 用于 Spark 大数据分析场景中的数据存储接口；ELB 用于将后端服务以公网入口暴露给外部访问。环境搭建过程不仅决定后续应用能否部署成功，也直接影响截图验收中关于 Kubernetes 版本、Worker 节点数量、节点状态和公网访问能力等关键评分点。

本项目在华为云华北-北京四区域开展实验,创建 CCE Standard 集群 `cloud-course-cce`，并配套创建 VPC、子网、Worker 节点和负载均衡资源。为了控制代金券消耗，同时满足任务书中不少于两个 Worker 节点和 Kubernetes 版本不低于 1.27 的要求，集群采用按需计费模式，Worker 节点规格选择 2 vCPU / 4 GiB。环境配置完成后，通过 CloudShell 连接集群并执行 `kubectl get nodes -o wide`、`kubectl cluster-info` 和 `kubectl version --client` 等命令进行验证。

2.1 云服务与资源配置

本项目使用的主要云服务及其作用如表 2 所示。可以看到，SWR、CCE、OBS 和 ELB 分别对应镜像管理、容器编排、数据存储和公网访问四个层面，共同构成了后续实验的基础环境。

表 2: 华为云实验环境资源概览

云服务	本项目用途	关键配置或说明
SWR	容器镜像仓库	保存 <code>backend:v1</code> 与 <code>frontend:v1</code> 镜像，为 CCE Deployment 提供镜像来源。
CCE	Kubernetes 集群	创建 <code>cloud-course-cce</code> 集群，版本为 v1.34，包含 2 个 Worker 节点。
OBS	Spark 数据存储接口	Spark 脚本支持通过 <code>s3a://</code> 路径读取数据；当前报告中 Spark 章节将结合数据分析脚本展开说明。
ELB	公网负载均衡	后端 <code>backend-svc</code> 通过共享型公网 ELB 暴露 <code>/api/ping</code> 接口。

2.1.1 SWR 镜像仓库配置

SWR 是本项目中连接本地容器构建与云端 Kubernetes 部署的关键环节。后端 Flask 服务和前端 Nginx 服务在本地完成 Docker 镜像构建后,需要推送至华为云 SWR,随后 CCE 节点才能在创建 Pod 时从私有镜像仓库拉取镜像。由于 SWR 镜像仓库通常为私有仓库,后续 Deployment 中还需要结合镜像拉取密钥或 CCE 自动生成的 default-secret 进行鉴权。

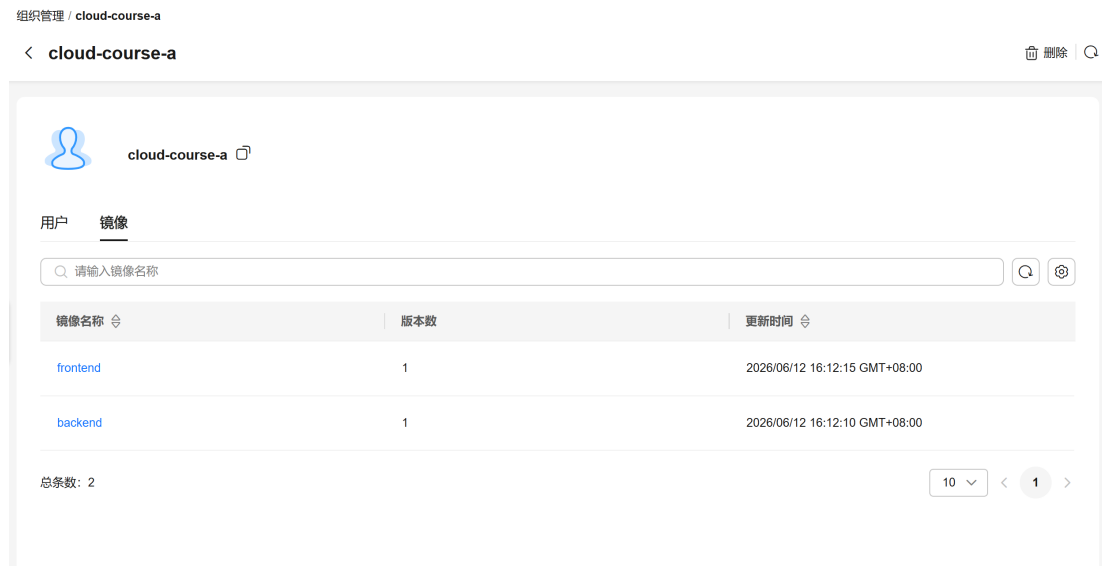


图 4: SWR 中 backend 与 frontend 镜像确认截图

图 4 展示了成员 A 的 SWR 组织中已经存在 backend 和 frontend 镜像,且镜像标签为 v1。该截图证明云端部署阶段使用的镜像不是临时占位配置,而是已经推送到华为云镜像仓库中的实际镜像资源。后续 T3 应用部署章节将进一步说明 Deployment 如何引用这些镜像地址,以及私有镜像拉取失败时如何通过 imagePullSecrets 解决鉴权问题。

2.1.2 CCE 集群配置

CCE 是第一部分云计算平台搭建的核心服务。项目创建的是 CCE Standard 集群,集群名称为 cloud-course-cce,区域为华北-北京四,Kubernetes 版本为 v1.34,满足任务书中“集群版本不低于 1.27”的要求。网络方面,集群绑定 VPC cloud-course-vpc,网段为 192.168.0.0/16;子网名称为 cloud-course-subnet,网段为 192.168.0.0/24。节点方面,集群创建 2 个 Worker 节点,节点规格为 c9.large.2,即 2 vCPU / 4 GiB,操作系统为 Ubuntu 22.04.5 LTS,容器运行时为 containerd。



图 5: 首次使用 CCE 时创建服务委托截图

图 5 显示了首次使用 CCE 时创建服务委托的过程。CCE 作为托管 Kubernetes 服务，需要通过服务委托访问部分云资源，例如负载均衡、存储和网络资源。该步骤是后续自动创建或绑定云端资源的前置条件。

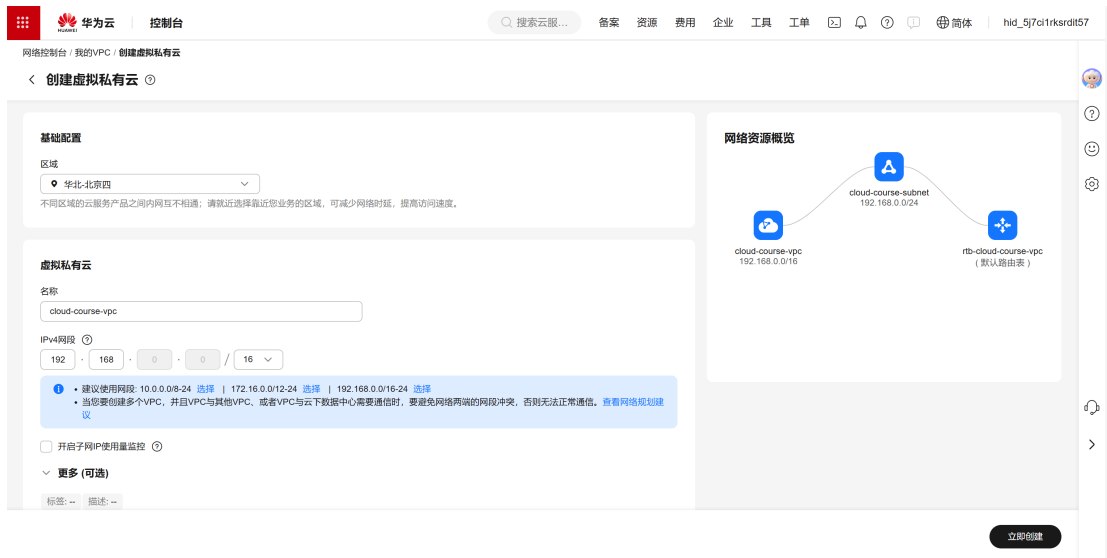


图 6: 创建 VPC 与子网配置页面截图

图 6 展示了 VPC 和子网的创建配置。VPC 为 CCE 节点、Pod 网络和云端负载均衡提供基础网络隔离环境，子网则决定 Worker 节点实际使用的内网地址范围。本项目使用统一命名的 `cloud-course-vpc` 和 `cloud-course-subnet`，便于后续资源定位和排错。

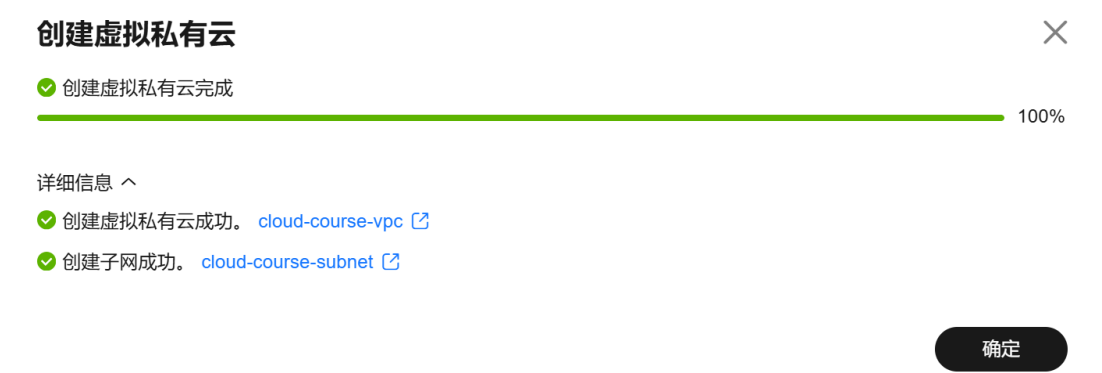


图 7: VPC 与子网创建成功截图

图 7 表明 VPC 和子网已经创建完成。该结果说明 CCE 集群创建前的网络基础资源已经就绪，后续 Worker 节点可以加入该网络并获得内网 IP。



图 8: VPC 列表中 cloud-course-vpc 状态可用截图

图 8 展示了 VPC 列表中 cloud-course-vpc 处于可用状态。该截图进一步证明网络资源创建成功，并且命名与本报告记录一致。

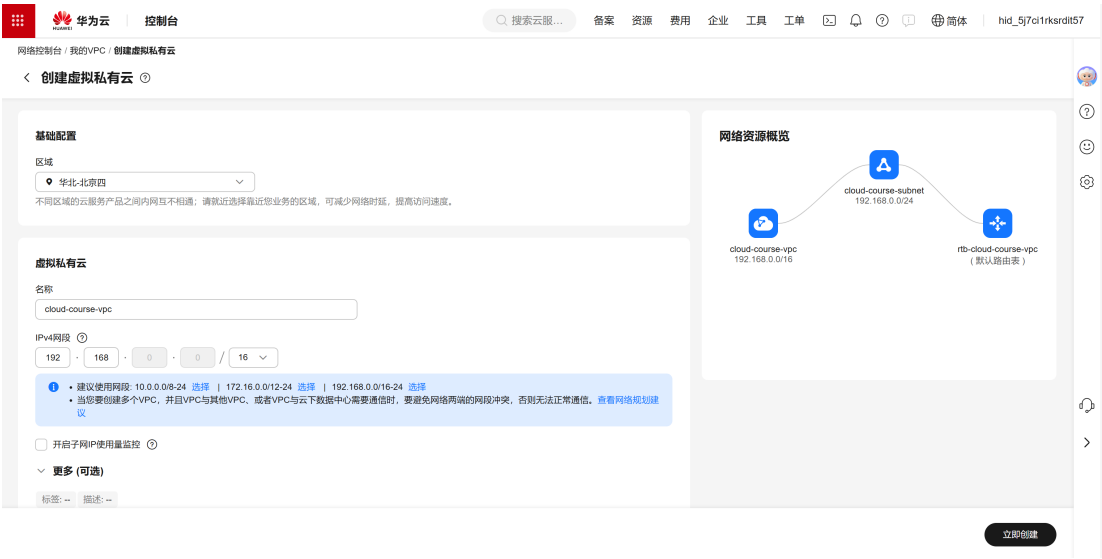


图 9: CCE 集群网络配置总览截图

图 9 展示了 CCE 集群创建时的网络配置总览。该页面用于确认集群绑定的 VPC、子网和容器网络配置是否正确，避免后续 Service、Pod 通信或 ELB 绑定出现网络层面的配置不一致。



图 10: CCE 网络配置中 VPC、子网和容器网络模型截图

图 10 进一步展示了 CCE 网络模型配置。该截图说明集群创建时已经明确选择 VPC、子网和容器网络模型，为后续 Pod 网络、Service 访问和集群内部通信提供基础。

配置概要

基础配置

集群类型：CCE Standard

计费模式：按需计费

集群名称：cloud-course-cce

集群委托：CCEAutoClusterAgency

集群版本：v1.34

控制节点架构：X86

集群最大管理规模：50 节点

集群 master 实例数：单实例

集群 master 实例分布策略：自动分配

网络配置

容器网络模型：VPC 网络

虚拟私有云：cloud-course-vpc (192.168.0.0/16)

默认节点子网：cloud-course-subnet (192.168.0.0/24)

启用IPv6：关闭

图 11: CCE 基础配置概要截图

图 11 展示了 CCE 基础配置概要。该截图用于确认集群名称、类型、版本等基础参数，尤其是 Kubernetes 版本信息，是任务书中版本要求的重要支撑材料。

配置概要

网络配置

容器网络模型：VPC 网络

虚拟私有云：cloud-course-vpc (192.168.0.0/16)

默认节点子网：cloud-course-subnet (192.168.0.0/24)

启用IPv6：关闭

节点默认安全组：自动生成

DataPlane V2：关闭

容器网段：自动设置网段

每节点预留的Pod IP数：128

服务网段(Service CIDR)：10.247.0.0/16

服务转发模式：iptables

安全服务

安全服务：未开启

高级配置

开启过载控制：开启

图 12: CCE 网络与安全服务配置概要截图

图 12 展示了网络与安全服务相关配置概要。该页面用于确认集群安全组、网络访

问和相关云服务配置，为后续节点加入集群和服务暴露提供保障。

配置概要

DataPlane V2：关闭

容器网段：自动设置网段

每节点预留的Pod IP数：128

服务网段(Service CIDR)：10.247.0.0/16

服务转发模式：iptables

安全服务

安全服务：未开启

高级配置

开启过载控制：开启

集群时区：(UTC+08:00) Asia/Shanghai

插件

	CCE容器网络插件 (Y...	默认安装
	CCE容器存储 (Evere...	默认安装
	CoreDNS域名解析	默认安装

图 13: CCE 网络与默认插件配置概要截图

图 13 记录了网络配置和默认插件配置概要。CCE 集群中的网络插件、CoreDNS

和存储插件会直接影响 Pod 之间通信、域名解析和 PVC 挂载能力，因此该截图是后续应用部署和 Redis 持久化验证的环境依据。



图 14: CCE 默认必要插件配置截图

图 14 展示了 CCE 集群创建时保留的默认必要插件。CoreDNS 用于集群内部服务名解析，Everest 存储插件用于后续 PVC 与云硬盘的对接，容器网络插件用于 Pod 网络通信。这些插件虽然不是业务应用代码的一部分，但它们决定了 Kubernetes 集群是否具备完整的基础能力。



图 15: 提交创建 CCE 集群前的最终配置确认截图

图 15 是提交创建 CCE 集群前的最终确认页面。该截图集中展示了集群创建前的关键参数，能够作为环境配置与报告记录一致性的证据。

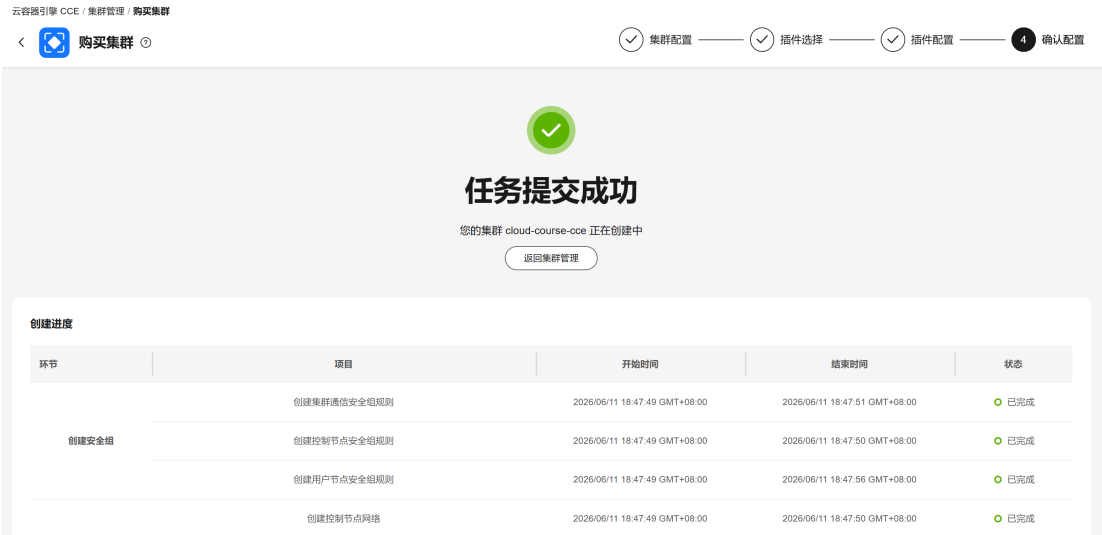


图 16: CCE 集群进入创建中状态截图

图 16 显示 CCE 集群创建任务已经提交并进入创建中状态。该截图记录了从配置确认到实际创建的过程，说明环境不是仅停留在配置草稿阶段。

2.1.3 OBS 数据存储配置

OBS 在任务书中主要服务于 Spark 大数据分析方向，用于保存待分析数据集并通过 s3a:// 接口供 Spark 作业读取。本项目的 spark/analysis.py 支持将数据路径作为参数传入，因此既可以读取仓库中的本地 douban_movies.csv，也可以在云端 Spark 环境中替换为 OBS 路径。由于当前截图文件夹中没有单独保存 OBS 控制台截图，本章不虚构 OBS 页面证据；后续 Spark 大数据分析章节将围绕数据读取方式、SparkApplication 配置和实际分析结果继续说明。

从架构角度看，OBS 与 CCE 的关系并不是直接部署关系，而是作为 Spark 作业的数据源存在。Spark Driver 和 Executor 在运行时通过 S3A 协议访问对象存储中的数据文件，完成 DataFrame 加载、清洗和 SQL 分析。该设计将计算资源和数据存储解耦，符合云计算环境中“存算分离”的基本思想。

2.1.4 ELB 负载均衡配置

ELB 用于对外暴露后端服务。在 Kubernetes 中,后端 Service 被定义为 LoadBalancer 类型，并通过华为云 ELB 获得公网访问入口。任务书要求通过 ELB 公网 IP 访问 /api/ping 并返回 {"status":"ok"}，因此 ELB 是否创建并正常绑定，直接关系到 T3 应用部署验收。

配置概要

区域	华北-北京四
名称	cloud-course-back end-elb
实例规格	
实例类型	共享型
网络配置	
网络类型	IPv4 公网,IPv4 私 网
所属VPC	cloud-course-vpc (192.168.0.0/16)
前端子网	cloud-course-subn et (192.168.0.0/24)
IPv4地址	自动分配IPv4地址
性能保障模式	默认开启

图 17 展示了共享型公网 ELB 的创建配置。该负载均衡器用于后端 `backend-svc` 的公网访问入口，后续通过 Service 注解将 ELB 与 Kubernetes Service 绑定。

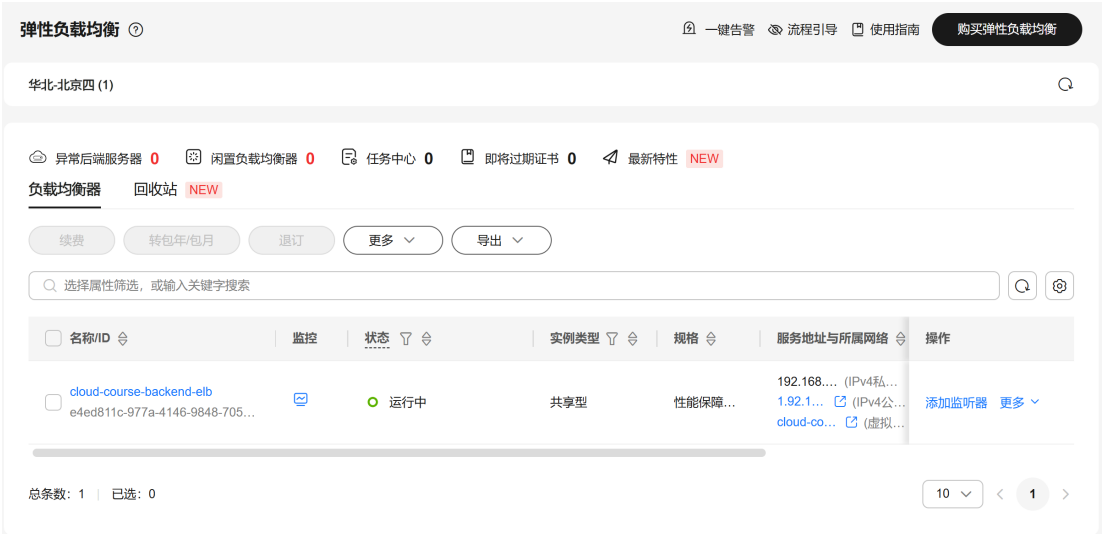


图 18: ELB 负载均衡器运行中状态截图

图 18 显示 ELB 资源已经创建并处于运行中状态。该状态说明公网负载均衡基础资源可用，为后续通过公网 IP 访问后端接口提供环境条件。

2.2 Kubernetes 集群信息

Kubernetes 集群信息是本章的重点，也是任务书中 T2 的直接验收内容。根据实验记录，集群创建完成后，通过 CCE 控制台和 CloudShell 分别进行了节点运行状态、kubectl 连接状态和客户端版本检查。相关环境参数如表 3 所示。

表 3: Kubernetes 集群关键参数

项目	内容
区域	华北-北京四
集群名称	cloud-course-cce
集群类型	CCE Standard
计费模式	按需计费
Kubernetes 版本	v1.34, 节点输出中 VERSION 为 v1.34.6-r0-34.0.23
Worker 节点数量	2 个
Worker 节点规格	c9.large.2, 2 vCPU / 4 GiB
操作系统	Ubuntu 22.04.5 LTS
容器运行时	containerd
VPC / 子网	cloud-course-vpc / cloud-course-subnet

2.2.1 集群版本与节点规格

集群版本和节点规格决定了后续 Kubernetes 资源是否能够稳定运行。本项目选择 v1.34 版本，显著高于任务书要求的 v1.27 下限；节点采用 2 vCPU / 4 GiB 规格，能够满足 Flask 后端、Redis、Nginx 前端以及基础 Spark Operator 验证的课程设计需求，同时控制云资源成本。

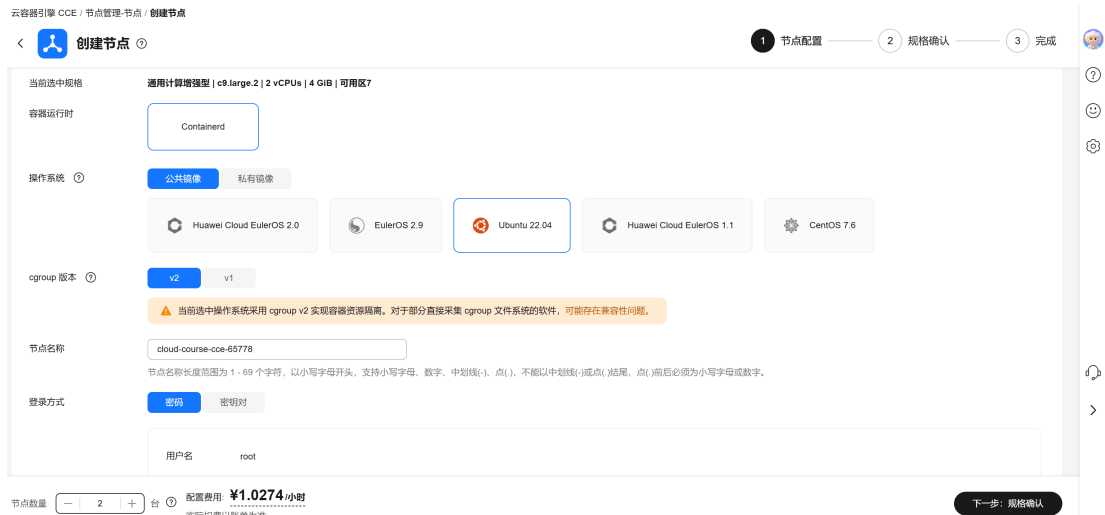


图 19: 创建 Worker 节点时的基础规格配置截图

图 19 展示了创建 Worker 节点时的基础规格配置。该截图用于证明节点创建不是采用默认未知配置，而是根据课程设计成本与运行需求选择了明确的实例规格。



图 20: 创建 Worker 节点时的存储与网络配置截图

图 20 展示了 Worker 节点创建过程中的存储与网络配置。该配置影响节点能否正常加入 VPC、能否创建容器运行环境，以及后续是否能够挂载存储卷。



图 21: Worker 节点规格确认页面截图

图 21 是 Worker 节点规格确认页面。该截图作为节点数量和规格的控制台证据，与后续 `kubectl get nodes -o wide` 的命令行证据相互补充。

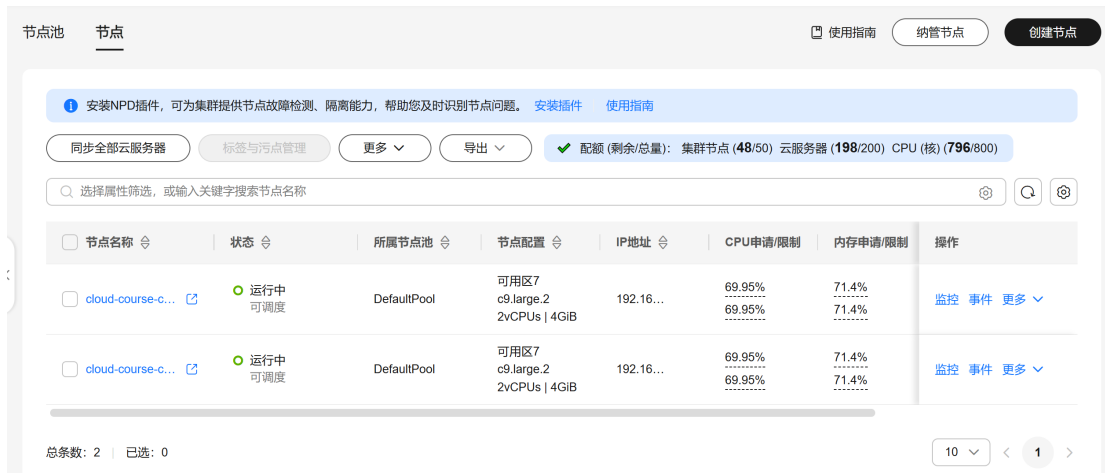


图 22: CCE 控制台显示两个 Worker 节点运行中截图

图 22 显示 CCE 控制台中两个 Worker 节点处于运行中状态。该截图从控制台视角证明节点已创建成功，而不仅是命令行输出中的临时状态。

2.2.2 kubectl 连接与节点状态

创建集群后，需要配置 `kubectl` 才能对 Kubernetes 集群执行资源管理操作。本项目使用华为云 CloudShell 配置 `kubectl`，并通过 `kubectl get nodes -o wide` 验证两个 Worker 节点均为 Ready。该命令输出同时包含节点名称、状态、角色、版本、内网 IP、操作系统和容器运行时等信息，是任务书 T2 的核心验收截图。

```

user@e8pgt9kmw5c5ycz-machine:~$ kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE   VERSION            INTERNAL-IP    EXTERNAL-IP    OS-IMAGE             KERNEL-VERSION
CONTAINER-RUNTIME
192.168.0.215                       Ready    <none>    75m   v1.34.6-r0-34.0.23  192.168.0.215  <none>         Ubuntu 22.04.5 LTS   5.15.0-173-generic
containerd://1.7.29-19-g707eaa844
192.168.0.237                       Ready    <none>    75m   v1.34.6-r0-34.0.23  192.168.0.237  <none>         Ubuntu 22.04.5 LTS   5.15.0-173-generic
containerd://1.7.29-19-g707eaa844
user@e8pgt9kmw5c5ycz-machine:~$

```

图 23: 核心验收: kubectl 查看两个 Worker 节点 Ready 状态截图

图 23 使用红框突出显示核心验收截图。截图中可以看到两个 Worker 节点的 STATUS 均为 Ready, VERSION 列为 v1.34.6-r0-34.0.23, 节点内网 IP 分别为 192.168.0.215 和 192.168.0.237。这说明集群节点已经成功加入 Kubernetes, 并且版本满足任务书要求。

```

user@ah2ggtculberhgg-machine:~$ kubectl cluster-info
Kubernetes control plane is running at https://10.198.58.100:5443
CoreDNS is running at https://10.198.58.100:5443/api/v1/namespaces/kube-system/services/coredns/dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
user@ah2ggtculberhgg-machine:~$

```

图 24: kubectl cluster-info 连接集群成功截图

图 24 展示了 kubectl cluster-info 的输出结果。该截图证明 kubectl 已经能够访问 Kubernetes 控制平面, 为后续通过 kubectl apply 部署 YAML 资源提供前提条件。

```

user@ah2ggtculberhgg-machine:~$ kubectl version --client
Client Version: v1.34.6-r0-34.0.3
Kustomize Version: v5.7.1
user@ah2ggtculberhgg-machine:~$

```

图 25: kubectl 客户端版本信息截图

图 25 展示了 CloudShell 中 kubectl 客户端版本信息。该截图说明实验环境中 kubectl 工具可用, 且后续对 Pod、Service、PVC 和 HPA 的管理均可通过命令行完成。

2.2.3 命名空间与资源规划

本项目基础任务主要使用 Kubernetes 的 default 命名空间, 便于课程设计验收时统一查看 Pod、Deployment、Service、ConfigMap、Secret、PVC 和 HPA 等资源。后端 Deployment 命名为 backend, Redis Deployment 命名为 redis, 前端 Deployment 命名为 frontend; 后端 Service 命名为 backend-svc, Redis Service 命名为 redis-svc, 前端 Service 命名为 frontend-svc。配置资源方面, 后端使用 backend-config 注入

Redis 地址和端口，使用 `redis-secret` 保存 Redis 密码字段；持久化存储方面，Redis 使用 `redis-data-pvc` 申请云硬盘存储。

资源规划遵循以下原则：第一，业务资源命名与任务书中的组件名称保持一致，便于截图识别和答辩说明；第二，敏感配置与非敏感配置分离，分别放入 `Secret` 和 `ConfigMap`；第三，对外访问只通过后端 `LoadBalancer Service` 暴露，Redis 保持 `ClusterIP` 内部访问，降低不必要的公网暴露风险；第四，后续 `Spark Operator` 或监控系统如需单独命名空间，将在对应章节中说明。本章环境配置完成后，后续章节即可在该 `Kubernetes` 集群之上继续完成应用部署、持久化、配置挂载、弹性伸缩和 `Spark` 大数据分析任务。

第三章 第一部分：云计算平台搭建

本章对应课程设计任务书第一部分“云计算平台搭建”，总分 50 分。该部分以“Flask 后端 API + Redis 数据库 + Nginx 前端页面”为应用对象，完成从本地容器化联调到华为云 CCE 集群部署的全过程。报告按照任务书 T1 至 T6 的顺序组织：T1 说明应用容器化与 SWR 镜像推送；T2 说明 CCE 集群搭建与节点 Ready 验证；T3 说明 `Kubernetes` 资源部署与公网接口访问；T4 说明 Redis PVC 持久化验证；T5 说明 Nginx 配置以 `ConfigMap Volume` 形式挂载；T6 说明 HPA 弹性伸缩配置及当前仍需补充的验收截图。

3.1 任务 1：应用容器化

任务 1 的目标是将后端和前端应用分别容器化，并通过本地联调与 SWR 推送证明镜像可运行、可分发。后端采用 Flask 提供 `/api/ping` 接口，并通过 Redis 客户端验证与 Redis 服务的连接状态；前端采用 Nginx 静态页面，页面中包含学生姓名和学号，满足验收识别要求。完成本地构建后，将 `backend:v1` 和 `frontend:v1` 推送至华为云 SWR，为后续 CCE 部署提供镜像来源。

3.1.1 后端 Dockerfile 多阶段构建

后端 Dockerfile 保留了任务书要求的多阶段构建结构。第一阶段基于 `python:3.11-slim` 安装依赖，并将依赖包安装到 `/build/packages`；第二阶段仍使用轻量级 Python 运行时镜像，只复制依赖包和应用代码，并通过 `PYTHONPATH=/app/packages` 指定依赖搜索路径。该结构将依赖安装过程与运行时环境分离，能够减少运行镜像中不必要的构建缓存和工具链内容。

本项目后端依赖文件中包含 `flask==3.0.0`、`redis==5.0.1` 和 `requests==2.31.0`。其中 `requests` 是在基础模板之外加入的自选 Python 包，满足任务书中“至少加入 1 个自选 Python 包”的要求。

3.1.2 后端依赖包修改与说明

后端服务依赖 Flask 提供 HTTP API，依赖 Redis Python 客户端连接 Redis 服务，并额外加入 requests 包以体现依赖扩展能力。虽然当前核心接口主要用于健康检查和 Redis 连接验证，但依赖文件采用固定版本号写法，能够减少镜像构建时由于上游包版本变化导致的不确定性。后端容器暴露 5000 端口，启动命令为 `python app.py`，与 Kubernetes Deployment 中的 `containerPort: 5000` 保持一致。

3.1.3 前端首页学号姓名标识

前端 Dockerfile 基于 `nginx:1.25-alpine`，将自定义 `nginx.conf` 复制到 `/etc/nginx/conf.d/default.conf` 并将 `static/` 下的静态页面复制到 Nginx 默认网页目录。前端首页 `index.html` 中写入了姓名“李欣纯”和学号“2023116052”，用于课程设计验收时识别提交者身份。该部分满足任务书中“在 Nginx 默认首页中加入本人学号和姓名”的要求。

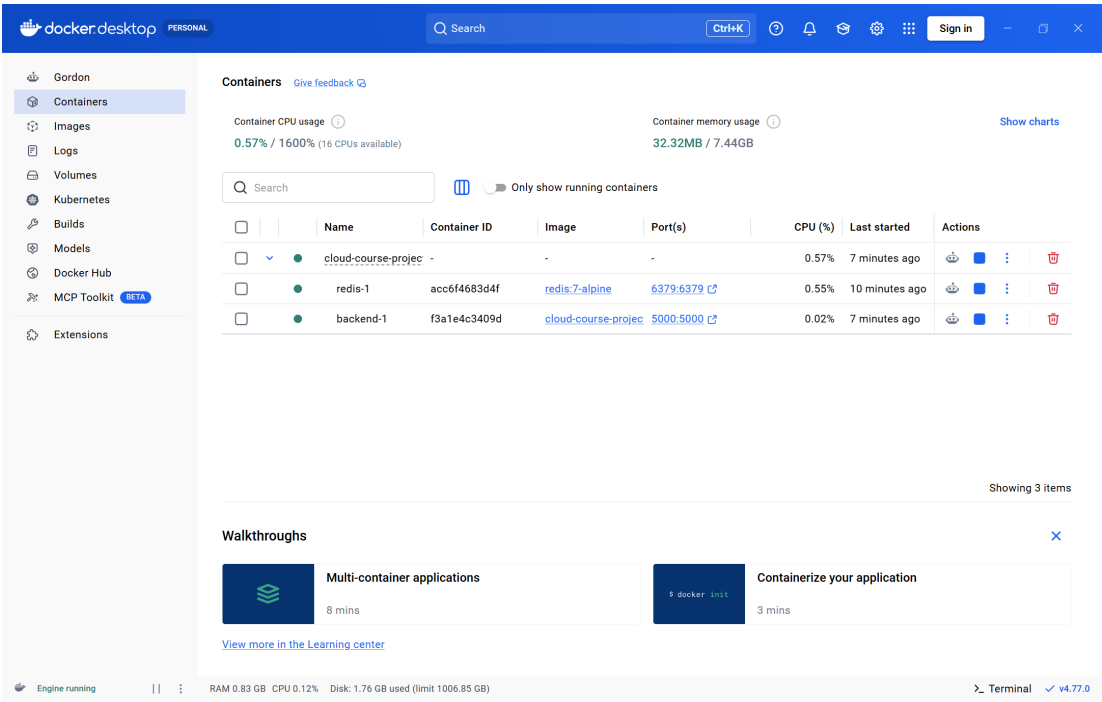


图 26: Docker Desktop 正常运行界面

图 26 显示 Docker Desktop 已正常运行。本地容器化实验依赖 Docker Engine 提供镜像构建和容器运行能力，该截图证明本地构建环境可用。

3.1.4 Docker Compose 本地联调验证

本地联调使用 `docker compose up -d --build` 同时启动后端与 Redis 服务。Compose 配置中后端通过环境变量读取 Redis 主机、端口和密码，Redis 使用 `redis:7-alpine` 镜像。联调重点不是仅验证容器能够启动，而是验证后端容器能够通过服务名访问 Redis，并且 `/api/ping` 接口能返回正常状态。

```

C:\WINDOWS\system32\cmd.exe:cloud-course-project
PS D:\cloud-course-project> docker compose up -d --build
time="2026-06-09T16:04:58+08:00" level=warning msg="D:\\cloud-course-project\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
#1 [internal] load local bake definitions
#1 reading from stdin 549B done
#1 DONE 0.0s

#2 [internal] load build definition from Dockerfile
#2 transferring dockerfile: 601B done
#2 DONE 0.0s

#3 [internal] load metadata for docker.io/library/python:3.11-slim
#3 DONE 0.0s

#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 93B done
#5 DONE 0.0s

#6 [builder 1/4] FROM docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9fc0e6a38295747e49ac0
#6 resolve docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9fc0e6a38295747e49ac0 0.0s done
#6 DONE 0.0s

#7 [builder 3/4] COPY requirements.txt .
#7 CACHED

#8 [stage-1 2/4] WORKDIR /app
#8 CACHED

#9 [builder 2/4] WORKDIR /build
#9 CACHED

#10 [builder 4/4] RUN pip install --no-cache-dir -r requirements.txt --target /build/packages
#10 CACHED

#11 [stage-1 3/4] COPY --from=builder /build/packages /app/packages
#11 CACHED

#12 [stage-1 4/4] COPY . .
#12 CACHED

#13 exporting to image
#13 exporting layers done
#13 exporting manifest sha256:6d4674a6a81f741ae27c9add732cfff00e9ee9191b6b523b719545499f30f4 done
#13 exporting config sha256:1836f3b2a270ccc2bf1225acec4f19c085fe85540b8b47fe9d0e506e5f8b95665 done
#13 exporting attestation manifest sha256:31d790267626b2950753772310a45f802e458963ad1db27c6405f043a52045 0.0s done
#13 exporting manifest list sha256:e670744770787b39fc5a50cd7c5e4901cb80030c58483098e82ebcb50b1ee0
#13 exporting manifest list sha256:e670744770787b39fc5a50cd7c5e4901cb80030c58483098e82ebcb50b1ee0 0.0s done
#13 naming to docker.io/library/cloud-course-project-backend:latest done
#13 unpacking to docker.io/library/cloud-course-project-backend:latest done
#13 DONE 0.1s

```

图 27: 执行 docker compose up 构建并启动服务截图

图 27 展示了执行 `docker compose up -d --build` 后服务启动的过程，说明本地镜像能够完成构建并以容器形式运行。

```

#3 [internal] load metadata for docker.io/library/python:3.11-slim
#3 DONE 0.0s

#4 [internal] load .dockerignore
#4 transferring context: 2B done
#4 DONE 0.0s

#5 [internal] load build context
#5 transferring context: 93B done
#5 DONE 0.0s

#6 [builder 1/4] FROM docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9ffc0e6e38295747e49ac0
#6 resolve docker.io/library/python:3.11-slim@sha256:a3ab0b966bc4e91546a033e22093cb840908979487a9ffc0e6e38295747e49ac0 0.0s done
#6 DONE 0.0s

#7 [builder 3/4] COPY requirements.txt .
#7 CACHED

#8 [stage-1 2/4] WORKDIR /app
#8 CACHED

#9 [builder 2/4] WORKDIR /build
#9 CACHED

#10 [builder 4/4] RUN pip install --no-cache-dir -r requirements.txt --target /build/packages
#10 CACHED

#11 [stage-1 3/4] COPY --from=builder /build/packages /app/packages
#11 CACHED

#12 [stage-1 4/4] COPY . .
#12 CACHED

#13 exporting to image
#13 exporting layers done
#13 exporting manifest sha256:6d4674e6a81f741ae27c9add732c99d06e9ce9191b6bf523fba19545499f80f4 done
#13 exporting config sha256:1685f82b2a70ccc2bf122ace4f13c065fe85540bb47fe9d0e066f8b8856655 done
#13 exporting attestation manifest sha256:31f90287c26b2950753722010e45f3902e458903ad1dbb12c76405f043ad2045 0.0s done
#13 exporting manifest list sha256:e57074477087b39fc5a50c47c5e490f1cb80030c5843098e2ebcb50bc1ee0
#13 exporting manifest list sha256:e57074477087b39fc5a50c47c5e490f1cb80030c5843098e2ebcb50bc1ee0 0.0s done
#13 naming to docker.io/library/cloud-course-project-backend:latest done
#13 unpacking to docker.io/library/cloud-course-project-backend:latest done
#13 DONE 0.1s

#14 resolving provenance for metadata file
#14 DONE 0.0s

[+] up 3/3
✔ Image cloud-course-project-backend Built 0.6s
✔ Container cloud-course-project-redis-1 Running 0.0s
✔ Container cloud-course-project-backend-1 Started 1.7s

What's next:
Filter, search, and stream logs from all your Compose services
in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs?appId=cloud-course-project
PS D:\cloud-course-project> docker compose ps

```

图 28: Docker Compose 服务运行状态截图

图 28 展示了本地 Compose 服务运行状态，后端和 Redis 容器均已启动，为接口访问和后端日志验证提供基础。

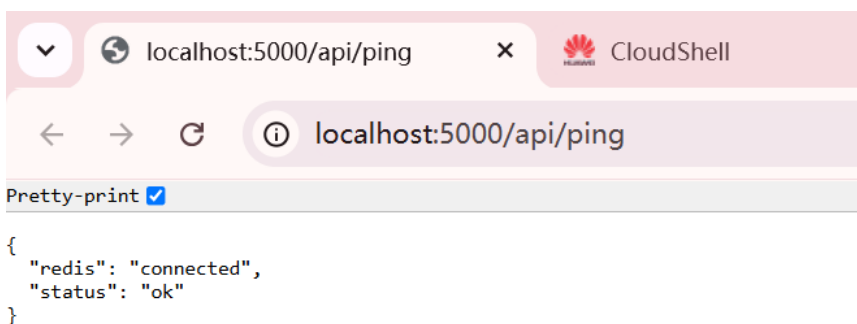


图 29: 本地访问后端接口返回结果截图

图 29 显示本地访问后端接口后获得响应结果，说明 Flask 后端服务能够对外提供 HTTP 接口。



图 30: 接口返回 status ok 验证截图

图 30 进一步显示接口返回中包含正常状态信息，说明后端健康检查接口可用，并为后续 CCE 中通过 ELB 访问 /api/ping 提供对照。

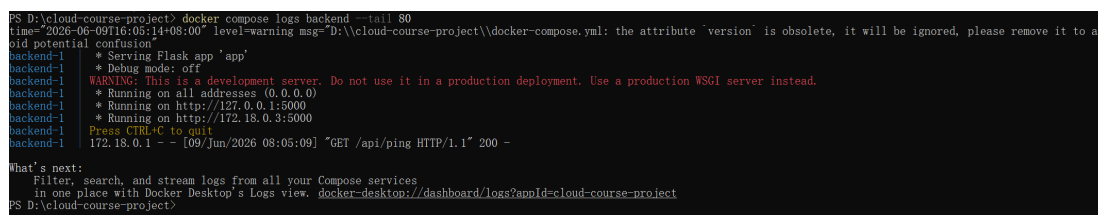


图 31: 后端日志显示收到请求截图

图 31 展示后端日志中收到请求的记录。该截图是本地联调的重要证据，说明浏览器或命令行访问确实到达了后端容器，而不仅是静态页面或缓存结果。

3.1.5 SWR 镜像推送与镜像 Tag 验证

本地联调通过后，将后端和前端镜像分别标记为 v1，并推送至华为云 SWR。镜像推送完成后，CCE Deployment 中即可引用 SWR 镜像地址创建 Pod。SWR 推送过程是从本地实验过渡到云端部署的关键步骤，若镜像未正确推送或标签不一致，后续 Kubernetes Pod 将无法拉取业务镜像。

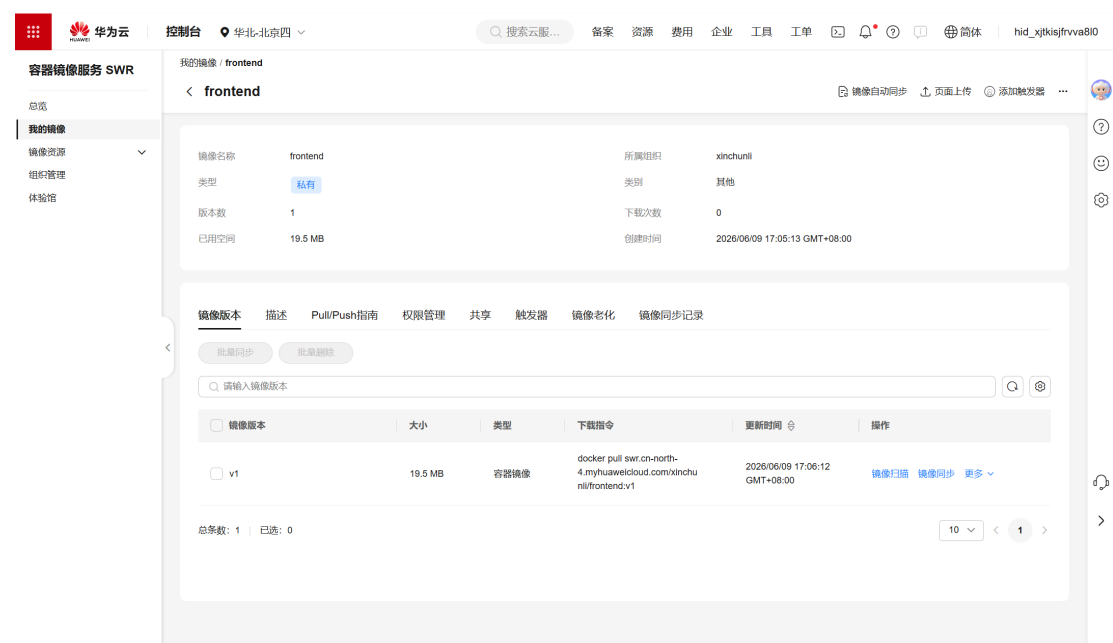


图 32: 后端镜像标记为 v1 截图

图 32 展示后端镜像完成 v1 标签标记。该标签与后续 Deployment 中引用的镜像标签保持一致。

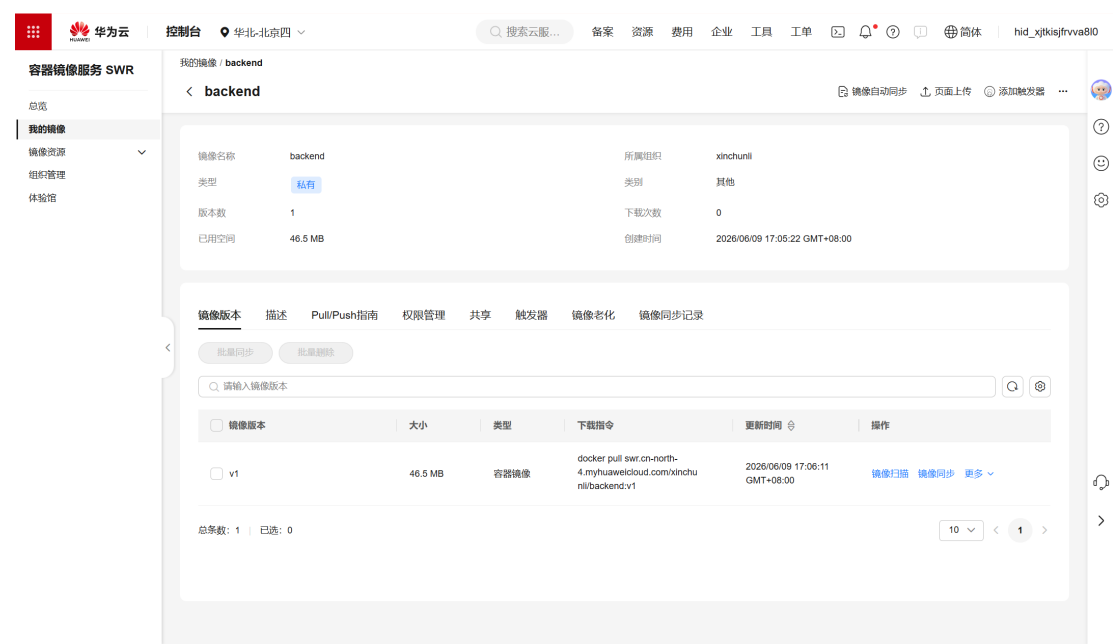


图 33: 前端镜像标记为 v1 截图

图 33 展示前端镜像完成 v1 标签标记，说明前端静态页面和 Nginx 配置已经形成可推送的镜像版本。

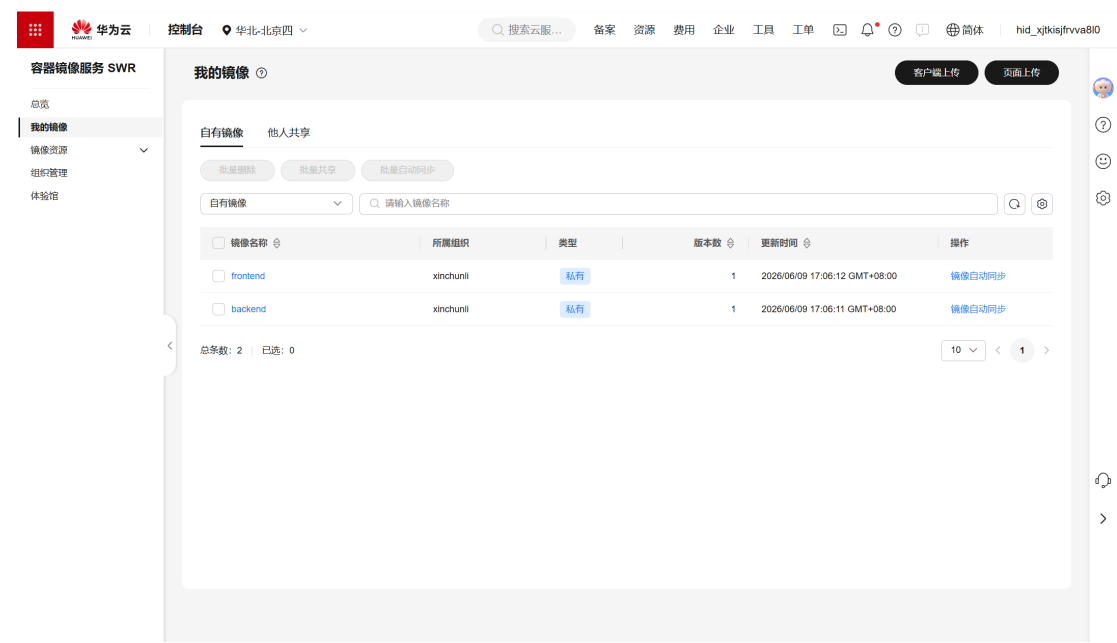


图 34: SWR 控制台镜像列表截图

图 34 显示 SWR 控制台中已经存在相关镜像及 Tag。该截图满足任务书中“SWR 控制台截图需含镜像名称和 Tag”的验收要求。

3.2 任务 2：CCE 集群搭建

任务 2 要求在华为云 CCE 控制台创建 Kubernetes 集群，并配置 kubectl，使至少两个 Worker 节点处于 Ready 状态。本项目已在第二章对 CCE、VPC、子网、Worker 节点、插件和 kubectl 连接过程进行了完整说明，并插入了 T2 的全部环境截图。本节从第一部分评分角度简要归纳验收结论。

3.2.1 CCE 集群创建过程

本项目在华为云华北-北京四区域创建 CCE Standard 集群 cloud-course-cce，并配套创建 VPC cloud-course-vpc 和子网 cloud-course-subnet。集群创建时保留 CCE 容器网络、CoreDNS 和 Everest 存储等默认必要插件，为后续服务发现、Pod 网络通信和 PVC 挂载提供基础能力。

3.2.2 Worker 节点 Ready 状态验证

集群创建完成后,使用 CloudShell 配置 kubectl,并执行 `kubectl get nodes -o wide`。第二章图 23 中可以看到两个 Worker 节点均处于 Ready 状态，说明节点已经成功加入集群并可承载业务 Pod。

3.2.3 Kubernetes 版本要求验证

节点输出中的 Kubernetes 版本为 v1.34.6-r0-34.0.23，高于任务书要求的 1.27 版本下限。节点操作系统为 Ubuntu 22.04.5 LTS，容器运行时为 containerd。由此可见，T2 在节点数量、节点状态和版本要求三个方面均满足验收标准。

3.3 任务 3：应用部署

任务 3 的目标是将任务 1 中完成的容器镜像部署到 CCE 集群中，并通过 Kubernetes 资源完成服务编排、配置注入、敏感信息管理和公网暴露。本项目部署的主要资源包括 backend Deployment、redis Deployment、frontend Deployment、backend-svc LoadBalancer Service、redis-svc ClusterIP Service、backend-config ConfigMap 和 redis-secret Secret。

```

user@jpx2i7z4h927mlx-machine:~$ kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE    VERSION    INTERNAL-IP    EXTERNAL-IP    OS-IMAGE    KERNEL-VERSION
192.168.0.215 Ready     <none>    16h    v1.34.6-r0-34.0.23    192.168.0.215    <none>         Ubuntu 22.04.5 LTS    5.15.0-173-generi
c containerd://1.7.29-19-g707eaa844
192.168.0.237 Ready     <none>    16h    v1.34.6-r0-34.0.23    192.168.0.237    <none>         Ubuntu 22.04.5 LTS    5.15.0-173-generi
c containerd://1.7.29-19-g707eaa844
user@jpx2i7z4h927mlx-machine:~$ kubectl get all
NAME                                TYPE    CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
service/kubernetes ClusterIP  10.247.0.1    <none>         443/TCP    20h
user@jpx2i7z4h927mlx-machine:~$ kubectl get pvc
No resources found in default namespace.
user@jpx2i7z4h927mlx-machine:~$ kubectl get configmap,secret
NAME                                DATA    AGE
configmap/kube-root-ca.crt         1        20h
secret/default-secret               1        20h
user@jpx2i7z4h927mlx-machine:~$

```

图 35: T3 部署前集群资源状态截图

图 35 展示应用部署前集群资源状态，用于说明部署操作是在已有 CCE 集群基础上进行，而非仅展示最终结果。

```

user@jpx2i7z4h927mlx-machine:~$ ls
bin  k8s.zip  projects  README.md
user@jpx2i7z4h927mlx-machine:~$ unzip k8s.zip
Archive:  k8s.zip
  creating: k8s/
  inflating: k8s/backend-config.yaml
  inflating: k8s/backend-deployment.yaml
  inflating: k8s/backend-hpa.yaml
  inflating: k8s/c1-pytorchjob.yaml
  inflating: k8s/c1-single-mnist-job.yaml
  inflating: k8s/frontend-deployment.yaml
  inflating: k8s/frontend-nginx-config.yaml
  inflating: k8s/redis-deployment.yaml
  inflating: k8s/redis-pvc.yaml
  inflating: k8s/redis-secret.yaml
  inflating: k8s/services.yaml
user@jpx2i7z4h927mlx-machine:~$ ls k8s
backend-config.yaml  c1-pytorchjob.yaml  frontend-nginx-config.yaml  redis-secret.yaml
backend-deployment.yaml  c1-single-mnist-job.yaml  redis-deployment.yaml      services.yaml
backend-hpa.yaml      frontend-deployment.yaml  redis-pvc.yaml
user@jpx2i7z4h927mlx-machine:~$

```

图 36: CloudShell 中 T3 YAML 文件准备完成截图

图 36 显示 CloudShell 中已准备好 T3 所需 YAML 文件。该截图证明部署使用的是可追溯的 Kubernetes 配置文件。

```
user@jpx2i7z4h927mlx-machine:~$ grep "image:" k8s/backend-deployment.yaml
    image: swr.cn-north-4.myhuaweicloud.com/cloud-course-a/backend:v1
user@jpx2i7z4h927mlx-machine:~$ grep "image:" k8s/frontend-deployment.yaml
    image: swr.cn-north-4.myhuaweicloud.com/cloud-course-a/frontend:v1
user@jpx2i7z4h927mlx-machine:~$ grep -n "replicas:" k8s/backend-deployment.yaml
7:   replicas: 2
user@jpx2i7z4h927mlx-machine:~$ grep -n "replicas:" k8s/redis-deployment.yaml
7:   replicas: 1
user@jpx2i7z4h927mlx-machine:~$ grep -n "type:" k8s/services.yaml
9:   type: LoadBalancer
22:  type: ClusterIP
35:  type: ClusterIP
user@jpx2i7z4h927mlx-machine:~$ grep -n "kubernetes.io/elb.class" k8s/services.yaml
7:   kubernetes.io/elb.class: union
user@jpx2i7z4h927mlx-machine:~$ grep -n "REDIS_HOST" k8s/backend-config.yaml
7:   REDIS_HOST: "redis-svc"
user@jpx2i7z4h927mlx-machine:~$
```

图 37: T3 关键 YAML 配置检查截图

图 37 展示了 T3 关键 YAML 配置检查结果，包括副本数、Service 类型、ELB 注解和 Redis 配置等内容，能够体现部署前对配置正确性的核对。

```
user@dhogkgjdvjsjy2ai-machine:~$ kubectl apply -f k8s/redis-pvc.yaml
persistentvolumeclaim/redis-data-pvc unchanged
user@dhogkgjdvjsjy2ai-machine:~$ kubectl apply -f k8s/backend-config.yaml
configmap/backend-config created
user@dhogkgjdvjsjy2ai-machine:~$ kubectl apply -f k8s/redis-secret.yaml
secret/redis-secret created
user@dhogkgjdvjsjy2ai-machine:~$ kubectl apply -f k8s/redis-deployment.yaml
deployment.apps/redis unchanged
user@dhogkgjdvjsjy2ai-machine:~$ kubectl apply -f k8s/backend-deployment.yaml
deployment.apps/backend created
user@dhogkgjdvjsjy2ai-machine:~$ kubectl apply -f k8s/services.yaml
service/backend-svc created
service/redis-svc created
service/frontend-svc created
user@dhogkgjdvjsjy2ai-machine:~$ kubectl apply -f k8s/frontend-nginx-config.yaml
configmap/frontend-nginx-config created
user@dhogkgjdvjsjy2ai-machine:~$ kubectl apply -f k8s/frontend-deployment.yaml
deployment.apps/frontend created
user@dhogkgjdvjsjy2ai-machine:~$
```

图 38: 应用 Kubernetes YAML 资源成功截图

图 38 显示 Kubernetes YAML 资源已成功应用到集群中，是 T3 从配置文件进入实际集群状态的直接证据。

3.3.1 后端 Deployment 配置

后端 Deployment 名称为 backend，副本数设置为 2，符合任务书“后端副本数=2”的要求。容器镜像来自华为云 SWR，容器端口为 5000，并配置了资源请求与限制，使调度器能够根据 CPU 和内存需求安排 Pod。后端通过 configMapRef 引入 Redis 地址和端口，通过 secretKeyRef 引入 Redis 密码字段，实现配置与代码分离。

3.3.2 Redis Deployment 配置

Redis Deployment 名称为 `redis`，副本数为 1，镜像为 `redis:7-alpine`。Redis 作为后端服务依赖的内部数据组件，不直接暴露公网访问入口，而是通过 ClusterIP Service 在集群内部提供访问。后续 T4 中 Redis 还会挂载 PVC，将数据目录持久化到云硬盘。

3.3.3 后端 LoadBalancer Service 配置

后端 Service 名称为 `backend-svc`，类型为 `LoadBalancer`，对外暴露 80 端口，并转发到后端容器的 5000 端口。由于华为云 CCE 中 LoadBalancer Service 需要与 ELB 资源绑定，本项目在遇到 EXTERNAL-IP 长时间 `<pending>` 后，创建共享型公网 ELB 并通过 Service 注解绑定，从而获得公网 IP。

```
user@dhogkgjdvjsjy2ai-machine:~$ kubectl get svc -o wide
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	SELECTOR
backend-svc	LoadBalancer	10.247.80.109	<pending>	80:30178/TCP	5m32s	app=backend
frontend-svc	ClusterIP	10.247.90.214	<none>	80/TCP	5m32s	app=frontend
kubernetes	ClusterIP	10.247.0.1	<none>	443/TCP	21h	<none>
redis-svc	ClusterIP	10.247.61.230	<none>	6379/TCP	5m32s	app=redis

图 39: backend-svc 外部 IP 持续 Pending 问题截图

图 39 记录了 `backend-svc` 初始外部 IP 处于 `<pending>` 的问题现象。该截图用于说明部署过程中存在真实排错过程，而不是简单套用模板。

```
user@dhogkgjdvjsjy2ai-machine:~$ kubectl annotate svc backend-svc kubernetes.io/elb.id=<e4ed811c-977a-4146-9848-705ab93f59a6> --overwrite
bash: e4ed811c-977a-4146-9848-705ab93f59a6: No such file or directory
user@dhogkgjdvjsjy2ai-machine:~$ kubectl annotate svc backend-svc kubernetes.io/elb.id=e4ed811c-977a-4146-9848-705ab93f59a6 --overwrite
service/backend-svc annotated
user@dhogkgjdvjsjy2ai-machine:~$ kubectl annotate svc backend-svc kubernetes.io/elb.class=union --overwrite
service/backend-svc annotated
user@dhogkgjdvjsjy2ai-machine:~$ kubectl get svc -o wide
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	SELECTOR
backend-svc	LoadBalancer	10.247.80.109	1.92.151.224,192.168.0.218	80:30178/TCP	20m	app=backend
frontend-svc	ClusterIP	10.247.90.214	<none>	80/TCP	20m	app=frontend
kubernetes	ClusterIP	10.247.0.1	<none>	443/TCP	21h	<none>
redis-svc	ClusterIP	10.247.61.230	<none>	6379/TCP	20m	app=redis

图 40: Service 列表与 ELB 公网 IP 截图

图 40 显示 `backend-svc` 为 `LoadBalancer` 类型，并已经获得 ELB 公网 IP。该截图说明公网访问入口已经建立。

3.3.4 Redis ClusterIP Service 配置

Redis Service 名称为 `redis-svc`，类型为 `ClusterIP`，端口为 6379。该设计符合最小暴露原则：Redis 只需要被后端 Pod 访问，不需要直接暴露到公网。后端通过 ConfigMap 中的 `REDIS_HOST=redis-svc` 和 `REDIS_PORT=6379` 访问 Redis 服务，体现 Kubernetes Service 名称解析和内部服务发现机制。

3.3.5 ConfigMap 与 Secret 配置

本项目使用 backend-config 保存非敏感配置，包括 Redis 主机名、端口和应用运行环境；使用 redis-secret 保存敏感密码字段。ConfigMap 与 Secret 分别承担非敏感配置和敏感配置管理职责，能够避免将环境相关配置写死在镜像中，也符合云原生应用“配置外置化”的实践原则。

```
user@dhogkgjdvjsjy2ai-machine:~$ kubectl get configmap,secret
NAME                                DATA  AGE
configmap/backend-config            3      29m
configmap/frontend-nginx-config    1      29m
configmap/kube-root-ca.crt         1      22h

NAME                                TYPE                                DATA  AGE
secret/default-secret              kubernetes.io/dockerconfigjson    1      22h
secret/redis-secret                Opaque                             1      29m
user@dhogkgjdvjsjy2ai-machine:~$
```

图 41: ConfigMap 与 Secret 资源存在截图

图 41 显示 backend-config、frontend-nginx-config 和 redis-secret 等资源已经存在，说明配置资源已经部署到集群中。

3.3.6 Pod 运行状态与接口访问验证

应用部署完成后，通过 kubectl get pods、浏览器访问和 curl 命令进行验证。核心验收点包括所有 Pod 处于 Running 状态，以及通过 ELB 公网 IP 访问 /api/ping 能够返回正常 JSON 结果。

```
user@dhogkgjdvjsjy2ai-machine:~$ kubectl get pods -o wide
NAME                                READY  STATUS   RESTARTS  AGE  IP            NODE             NOMINATED NODE  READINESS GATES
backend-5766d749f9-8pmp1           1/1    Running  0          24s  172.16.0.135  192.168.0.237    <none>          <none>
backend-5766d749f9-dtlg5           1/1    Running  0          28s  172.16.0.7    192.168.0.215    <none>          <none>
frontend-b8659fff8-dwz87          1/1    Running  0          22s  172.16.0.6    192.168.0.215    <none>          <none>
redis-79d8ffbd4c-gzcz6            1/1    Running  0          53m  172.16.0.4    192.168.0.215    <none>          <none>
user@dhogkgjdvjsjy2ai-machine:~$
```

图 42: 核心验收：Pod 全部 Running 状态截图

图 42 显示 backend、frontend 和 redis 相关 Pod 均处于 Running 状态，其中后端有两个副本，符合任务书要求。

```
user@dhogkgjdvjsjy2ai-machine:~$ kubectl get deploy -o wide
NAME    READY  UP-TO-DATE  AVAILABLE  AGE  CONTAINERS  IMAGES
backend 2/2    2            2           4m42s backend     swr.cn-north-4.myhuaweicloud.com/cloud-course-a/backend:v1
frontend 1/1    1            1           4m39s frontend   swr.cn-north-4.myhuaweicloud.com/cloud-course-a/frontend:v1
redis   1/1    1            1           65m  redis       redis:7-alpine
user@dhogkgjdvjsjy2ai-machine:~$
```

图 43: Deployment 副本数与镜像信息截图

图 43 展示了 Deployment 副本数和镜像信息，可以看到 backend、frontend 和 redis 的部署状态以及镜像来源。

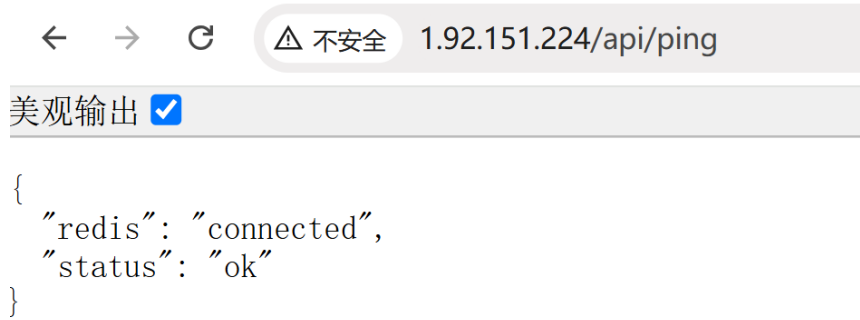


图 44: 浏览器访问 api/ping 返回正常截图

图 44 显示通过浏览器访问公网入口后，后端接口返回正常，说明 ELB 到后端 Service 的访问路径可用。

```
user@dhogkgjdvjsjy2ai-machine:~$ curl http://1.92.151.224/api/ping
{"redis":"connected","status":"ok"}
user@dhogkgjdvjsjy2ai-machine:~$
```

图 45: curl 访问 api/ping 返回正常截图

图 45 显示命令行访问 `/api/ping` 返回 `{"redis":"connected","status":"ok"}`。该结果不仅说明后端接口可访问，也说明后端能够连接 Redis。

```
user@dhogkgjdvjsjy2ai-machine:~$ kubectl get pods -l app=backend
NAME                                READY    STATUS    RESTARTS   AGE
backend-5766d749f9-8pmp1            1/1      Running   0           23m
backend-5766d749f9-dtlg5            1/1      Running   0           23m
user@dhogkgjdvjsjy2ai-machine:~$ kubectl logs backend-5766d749f9-8pmp1 --tail=50
172.16.0.129 - - [12/Jun/2026 08:44:30] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:44:45] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:45:00] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:45:15] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:45:30] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:45:45] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:46:00] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:46:15] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:46:30] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:46:45] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:47:00] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:47:15] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:47:30] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:47:45] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:48:00] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:48:15] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:48:30] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:48:45] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:49:00] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:49:15] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:49:30] "GET /api/ping HTTP/1.1" 200 -
192.168.0.215 - - [12/Jun/2026 08:49:33] "GET http://ossweb-img.qq.com/images/js/eas/eas.js HTTP/1.1" 404 -
172.16.0.129 - - [12/Jun/2026 08:49:45] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:50:00] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:50:15] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:50:30] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:50:45] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:51:00] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:51:15] "GET /api/ping HTTP/1.1" 200 -
172.16.0.129 - - [12/Jun/2026 08:51:30] "GET /api/ping HTTP/1.1" 200 -
```

图 46: 后端 Pod 日志收到请求截图

图 46 展示后端 Pod 日志收到请求并返回 200 状态码，进一步证明公网访问请求最终到达后端业务容器。

3.4 任务 4：持久化存储

任务 4 要求为 Redis 配置 PVC，并验证 Redis Pod 删除重建后数据不丢失。本项目创建 `redis-data-pvc`，使用华为云 `csi-disk` StorageClass 申请 10Gi 云硬盘，并在 Redis Deployment 中将 PVC 挂载到容器内 `/data` 目录。随后通过写入 `testkey=hello`、删除 Redis Pod、等待新 Pod 重建、再次查询 `testkey` 的方式验证持久化效果。

```
user@jpx2i7z4h927mlx-machine:~$ kubectl apply -f k8s/redis-pvc.yaml
persistentvolumeclaim/redis-data-pvc created
user@jpx2i7z4h927mlx-machine:~$ kubectl apply -f k8s/redis-deployment.yaml
deployment.apps/redis created
user@jpx2i7z4h927mlx-machine:~$
```

图 47: Redis PVC 与 Deployment 创建成功截图

图 47 显示 Redis PVC 和 Deployment 配置已经成功应用到集群中。

3.4.1 Redis PVC 创建与绑定

PVC 是 Pod 与底层云硬盘之间的抽象层。通过 PVC，Redis Pod 不直接绑定某一个固定节点上的本地目录，而是使用 Kubernetes 和云平台共同管理的持久化存储资源。这样即使 Pod 被删除并在同一节点或其他可用位置重建，数据仍保存在 PVC 关联的卷中。

```
user@jpx2i7z4h927mlx-machine:~$ kubectl get pvc
NAME          STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
redis-data-pvc Bound      pvc-76ec1040-9400-4d66-aafd-5a26fc7d5d00  10Gi       RWO            csi-disk       <unset>                 54s
```

图 48: 核心验收：Redis PVC Bound 状态截图

图 48 显示 `redis-data-pvc` 状态为 `Bound`，容量为 10Gi，StorageClass 为 `csi-disk`。这说明 PVC 已成功绑定到底层存储资源。

3.4.2 Redis Deployment 挂载 PVC

Redis Deployment 中定义了名为 `redis-data` 的 volume，并通过 `persistentVolumeClaim` 引用 `redis-data-pvc`；容器中通过 `volumeMounts` 将该卷挂载到 `/data`。Redis 的数据目录因此不再只存在于容器临时文件系统中，而是落到 PVC 对应的持久化存储上。

```

Ready:          True
Restart Count:  0
Limits:
  cpu:          300m
  memory:       512Mi
Requests:
  cpu:          100m
  memory:       128Mi
Environment:    <none>
Mounts:
  /data from redis-data (rw)
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-74fw5 (ro)
Conditions:
  Type                               Status
  PodReadyToStartContainers          True
  Initialized                        True
  Ready                              True
  ContainersReady                    True
  PodScheduled                       True
Volumes:
  redis-data:
    Type:          PersistentVolumeClaim (a reference to a PersistentVolumeClaim in the same namespace)
    ClaimName:     redis-data-pvc
    ReadOnly:      false
  kube-api-access-74fw5:
    Type:          Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds: 3607
    ConfigMapName:  kube-root-ca.crt
    Optional:      false
    DownwardAPI:   true
QoS Class:       Burstable
Node-Selectors:  <none>
Tolerations:     node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                  node.kubernetes.io/unreachable:NoExecute op=Exists for 300s

```

图 49: Redis Pod 挂载 PVC 详情截图

图 49 展示 Redis Pod 的挂载详情，可以看到 `/data` 目录与 PVC 的关联关系，证明 Redis 数据目录已经通过 Volume 挂载到持久化卷。

3.4.3 Redis 测试数据写入

持久化验证首先需要向 Redis 写入测试数据。实验中进入 Redis Pod，执行 `redis-cli SET testkey` 写入键值，并立即执行 `GET testkey` 验证写入成功。

```

user@jpx2i7z4h927mlx-machine:~$ kubectl exec -it $REDIS_POD -- redis-cli SET testkey "hello"
OK
user@jpx2i7z4h927mlx-machine:~$ kubectl exec -it $REDIS_POD -- redis-cli GET testkey
"hello"
user@jpx2i7z4h927mlx-machine:~$

```

图 50: 核心验收：Redis 写入 testkey 成功截图

图 50 显示 Redis 写入 `testkey` 返回 `OK`，并且读取结果为 `hello`。这为后续 Pod 删除后的数据对比提供基准。

3.4.4 Redis Pod 删除与自动重建

为了模拟 Pod 故障或重建场景，实验中删除当前 Redis Pod。由于 Redis 由 Deployment 管理，删除 Pod 后控制器会自动创建新的 Redis Pod，使实际副本数重新满足期望状态。

```
user@jpx2i7z4h927mlx-machine:~$ kubectl delete pod $REDIS_POD
pod "redis-79d8ffbd4c-hq8bg" deleted from default namespace
```

图 51: 核心验收：删除 Redis Pod 触发重建截图

图 51 显示旧 Redis Pod 被删除，Deployment 控制器开始触发 Pod 重建过程。

```
user@jpx2i7z4h927mlx-machine:~$ kubectl get pods -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP          NODE           NOMINATED NODE   READINESS GATES
redis-79d8ffbd4c-gzc6 1/1     Running   0          96s   172.16.0.4   192.168.0.215   <none>           <none>
```

图 52: Redis Pod 重建后 Running 状态截图

图 52 显示新的 Redis Pod 已重新创建并处于 Running 状态，说明 Deployment 自愈机制生效。

3.4.5 重建后数据持久化验证

新 Redis Pod 启动后，再次执行 GET testkey。如果 Redis 数据只存在于旧 Pod 的临时文件系统中，则 Pod 删除后该键值会丢失；若 PVC 挂载生效，则新 Pod 仍能从 /data 目录读取到原数据。

```
user@jpx2i7z4h927mlx-machine:~$ NEW_REDIS_POD=$(kubectl get pod -l app=redis -o jsonpath='{.items[0].metadata.name}')
user@jpx2i7z4h927mlx-machine:~$ kubectl exec -it $NEW_REDIS_POD -- redis-cli GET testkey
"hello"
```

图 53: 核心验收：重建后查询 testkey 仍返回 hello 截图

图 53 显示新 Pod 中再次查询 testkey 仍返回 hello。这说明 Redis 数据没有随着 Pod 删除而丢失，PVC 持久化存储配置生效。

3.5 任务 5: ConfigMap Volume 挂载

任务 5 要求将 Nginx 反向代理配置以 ConfigMap Volume 的方式挂载到前端 Pod，而不是通过环境变量传递。该任务的核心是证明配置文件来自 ConfigMap，并且修改 ConfigMap 后能够在 Pod 内文件系统中验证更新结果。本项目使用 frontend-nginx-config ConfigMap 保存 Nginx default.conf，并在 frontend Deployment 中将其挂载到 /etc/nginx/conf.d/

3.5.1 Nginx 配置 ConfigMap 创建

Nginx 配置文件包含静态页面根目录、默认首页以及 /api/ 反向代理规则。通过 ConfigMap 保存 Nginx 配置，可以在不重建前端镜像的情况下调整代理目标或新增配置注释，体现配置与镜像分离。

```
user@ipue24kxppqgza4-machine:~$ kubectl describe configmap frontend-nginx-config
Name:      frontend-nginx-config
Namespace: default
Labels:    <none>
Annotations: <none>

Data
====
default.conf:
-----
server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }

    location /api/ {
        proxy_pass http://backend-svc:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

BinaryData
====
Events: <none>
user@ipue24kxppqgza4-machine:~$
```

图 54: Nginx 配置 ConfigMap 内容截图

图 54 展示 frontend-nginx-config 中保存的 Nginx 配置内容，说明配置文件已经以 Kubernetes ConfigMap 的形式存在。

3.5.2 前端 Deployment 挂载 ConfigMap

frontend Deployment 中定义了 nginx-config volume, volume 来源为 frontend-nginx-config, 并通过 volumeMounts 挂载到 Nginx 默认配置路径。由于本项目使用 subPath 挂载单个 default.conf 文件, 修改 ConfigMap 后需要通过 `kubectl rollout restart deployment frontend` 创建新 Pod, 使新 Pod 在启动时重新挂载最新配置。

```

user@ipue24kxppqgza4-machine:~$ kubectl describe deploy frontend
Name: frontend
Namespace: default
CreationTimestamp: Fri, 12 Jun 2026 08:29:13 +0000
Labels: <none>
Annotations: deployment.kubernetes.io/revision: 2
Selector: app=frontend
Replicas: 1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels: app=frontend
  Containers:
    frontend:
      Image: swr.cn-north-4.myhuaweicloud.com/cloud-course-a/frontend:v1
      Port: 80/TCP
      Host Port: 0/TCP
      Limits:
        cpu: 200m
        memory: 128Mi
      Requests:
        cpu: 50m
        memory: 64Mi
      Environment: <none>
      Mounts:
        /etc/nginx/conf.d/default.conf from nginx-config (rw,path="default.conf")
  Volumes:
    nginx-config:
      Type: ConfigMap (a volume populated by a ConfigMap)
      Name: frontend-nginx-config
      Optional: false
      Node-Selectors: <none>
      Tolerations: <none>
  Conditions:
    Type          Status  Reason

```

图 55: 前端 Deployment 挂载 ConfigMap 基本信息截图

图 55 展示 frontend Deployment 的镜像、挂载路径等信息，说明前端容器使用 ConfigMap 提供 Nginx 配置。

```

Conditions:
  Type          Status  Reason
  ----          -
  Available     True    MinimumReplicasAvailable
  Progressing   True    NewReplicaSetAvailable
OldReplicaSets: frontend-5789cccb98 (0/0 replicas created)
NewReplicaSet:  frontend-b8659fff8 (1/1 replicas created)
Events:         <none>
user@ipue24kxppqgza4-machine:~$ █

```

图 56: 前端 Deployment Volume 来源为 ConfigMap 截图

图 56 进一步显示 volume 来源为 frontend-nginx-config，证明挂载关系并非普通空目录或镜像内置文件。

3.5.3 配置修改后的 Pod 内文件验证

实验首先进入 frontend Pod 查看默认 Nginx 配置文件，随后修改 ConfigMap，在配置中加入 # T5 ConfigMap volume update verified 标记，重新 apply 并滚动重启

frontend Deployment。最后再次进入新 Pod，查看 `/etc/nginx/conf.d/default.conf`，验证配置文件内容已经更新。

```
user@ipue24kxppqgza4-machine:~$ FRONTEND_POD=$(kubectl get pod -l app=frontend -o jsonpath='{.items[0].metadata.name}')
user@ipue24kxppqgza4-machine:~$ kubectl exec $FRONTEND_POD -- cat /etc/nginx/conf.d/default.conf
server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }

    location /api/ {
        proxy_pass http://backend-svc:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

图 57: 修改前 exec 查看默认 Nginx 配置文件截图

图 57 显示修改前 Pod 内 Nginx 配置文件内容，用于作为 ConfigMap 更新前的对照。

```
user@ipue24kxppqgza4-machine:~$ cat > /tmp/default.conf <<'EOF'
> server {
>     listen 80;
>
>     # T5 ConfigMap volume update verified
>
>     location / {
>         root /usr/share/nginx/html;
>         index index.html;
>     }
>
>     location /api/ {
>         proxy_pass http://backend-svc:80;
>         proxy_set_header Host $host;
>         proxy_set_header X-Real-IP $remote_addr;
>     }
> }
> EOF
user@ipue24kxppqgza4-machine:~$
user@ipue24kxppqgza4-machine:~$ kubectl create configmap frontend-nginx-config \
> --from-file=default.conf=/tmp/default.conf \
> -o yaml --dry-run=client | kubectl apply -f -
configmap/frontend-nginx-config configured
user@ipue24kxppqgza4-machine:~$
```

图 58: 修改 ConfigMap 后重新 apply 成功截图

图 58 显示修改后的 ConfigMap 已重新应用到集群中。

```

user@ipue24kxppqgza4-machine:~$ kubectl rollout restart deployment frontend
deployment.apps/frontend restarted
user@ipue24kxppqgza4-machine:~$ kubectl rollout status deployment frontend
Waiting for deployment "frontend" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "frontend" rollout to finish: 1 old replicas are pending termination...
deployment "frontend" successfully rolled out
user@ipue24kxppqgza4-machine:~$ FRONTEND_POD=$(kubectl get pod -l app=frontend -o jsonpath='{.items[0].metadata.name}')
user@ipue24kxppqgza4-machine:~$ kubectl exec $FRONTEND_POD -- cat /etc/nginx/conf.d/default.conf
server {
    listen 80;

    # T5 ConfigMap volume update verified

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }

    location /api/ {
        proxy_pass http://backend-svc:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
user@ipue24kxppqgza4-machine:~$

```

图 59: 核心验收: 修改后 exec 验证配置文件已更新截图

图 59 显示 frontend Deployment 滚动重启后, 新 Pod 内的 default.conf 已包含更新标记, 说明 ConfigMap Volume 挂载和配置更新验证成功。

```

user@ipue24kxppqgza4-machine:~$ kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE             NOMINATED NODE   READINESS GATES
backend-5766d749f9-8pmp1            1/1     Running   0           3h41m  172.16.0.135    192.168.0.237    <none>           <none>
backend-5766d749f9-dtlg5            1/1     Running   0           3h41m  172.16.0.7      192.168.0.215    <none>           <none>
frontend-5c77bd758d-46ls5           1/1     Running   0           2m9s   172.16.0.136    192.168.0.237    <none>           <none>
redis-79d8ffbd4c-gzcz6              1/1     Running   0           4h34m  172.16.0.4      192.168.0.215    <none>           <none>
user@ipue24kxppqgza4-machine:~$

```

图 60: ConfigMap 更新后 Pod 全部 Running 截图

图 60 显示 ConfigMap 更新和前端滚动重启后, backend、frontend 和 redis Pod 均处于 Running 状态, 说明配置更新没有破坏应用整体运行状态。

3.5.4 Volume 挂载与 envFrom 适用场景对比

ConfigMap 通过 envFrom 注入时, 适合传递短小的键值型配置, 例如 Redis 主机名、端口、运行环境等; 应用读取这些配置通常依赖环境变量, 形式简单, 适用于启动参数和少量配置项。ConfigMap 通过 Volume 挂载时, 适合传递完整配置文件, 例如 Nginx 配置、应用配置模板或多行脚本; 应用可以像读取普通文件一样读取配置内容, 结构更完整, 也更适合复杂文本配置。

本项目中后端 Redis 地址和端口属于典型键值配置, 因此使用 ConfigMap 环境变量注入; 前端 Nginx 反向代理配置属于完整文件, 因此使用 ConfigMap Volume 挂载更合适。二者共同体现了 Kubernetes 配置外置化的两种常见方式。

3.6 任务 6: HPA 弹性伸缩

任务 6 要求为后端 Deployment 配置 HPA, 并通过压测证明后端 Pod 能够从 1 个扩展到 2 个或更多, 停止压测后再缩回 1 个。当前仓库中已经存在 k8s/backend-hpa.yaml, 但未发现 T6 对应的 metrics-server、HPA 状态、压测、扩容和缩容截图。因此, 本节只

基于现有 YAML 文件说明 HPA 配置设计，并明确列出后续需要补充的验收材料，不对尚未提供截图的实验结果进行虚构。

3.6.1 metrics-server 可用性验证

HPA 依赖 metrics-server 提供 CPU、内存等指标数据。任务书要求先执行 `kubectl top nodes` 验证 metrics-server 可用。当前文件夹中没有 `kubectl top nodes` 截图，因此该项截图需要后续补充。建议补充截图时确保输出中包含节点名称、CPU 使用量、CPU 使用率、内存使用量和内存使用率，以证明 HPA 能够获取资源指标。

3.6.2 后端 HPA 资源配置

现有 `backend-hpa.yaml` 使用 `autoscaling/v2` API，HPA 名称为 `backend-hpa`，作用目标为 `backend Deployment`。配置中 `minReplicas=1`、`maxReplicas=4`，CPU 平均利用率目标为 60%。这与任务书要求一致，表示当后端 Pod 的平均 CPU 利用率持续超过目标阈值时，HPA 控制器会增加副本数，最高扩展到 4 个。

3.6.3 压测方法与扩容过程截图

任务书推荐使用 `ab -n 10000 -c 200 http://<ELB_IP>/api/ping` 对后端接口发起并发压测，并在另一个终端中执行 `kubectl get pods -w` 观察后端 Pod 扩容过程。当前文件夹中没有压测过程截图和 Pod 扩容截图，因此该项验收材料仍需补充。补充时建议至少保留两类截图：一是压测命令及其并发请求输出；二是 `kubectl get pods -w` 中 `backend Pod` 数量从 1 增加到 2 个或更多的时刻。

3.6.4 停止压测后的缩容过程截图

停止压测后，HPA 不会立即缩容，而是需要等待指标下降、控制器重新评估以及稳定窗口结束。当前文件夹中没有停止压测后的缩容截图，因此该项也需补充。建议截图中显示 `backend Pod` 数量回落到 1，并同时保留 `kubectl get hpa` 或 `kubectl describe hpa backend` 输出，用于解释当前副本数、目标 CPU 利用率和事件记录。

3.6.5 扩容延迟、冷却时间与降本价值分析

HPA 扩容存在延迟，主要原因包括 metrics-server 指标采集周期、HPA 控制器评估周期、Deployment 创建新 Pod 的调度与镜像启动时间等。缩容通常比扩容更保守，这是为了避免短时间指标波动导致副本数频繁上下抖动。对于课程设计中的后端服务而言，HPA 的价值在于高负载时自动增加副本保障服务可用性，低负载时自动减少副本降低资源占用和云资源成本。

需要说明的是，由于当前仓库未提供 T6 实际压测和扩缩容截图，本报告在本节中不将 HPA 判定为已完成验收。后续若补充 `kubectl top nodes`、`kubectl get hpa`、压测输出、扩容截图和缩容截图，即可将本节从“配置说明”完善为“完整验收记录”。

第四章 第二部分：Spark 大数据分析

4.1 A-0: Spark 环境部署

为了在 Kubernetes 环境中运行 Spark 作业，本项目采用 Spark Operator 方式部署 Spark 集群。Spark Operator 是 Kubernetes 环境下用于管理 SparkApplication 资源的控制器，能够自动完成 Driver Pod、Executor Pod 的创建、监控与销毁，从而实现 Spark 作业的声明式管理。相比传统 `spark-submit` 方式，Spark Operator 更符合云原生架构思想，便于与 Kubernetes 的调度、资源管理和监控体系进行集成。

4.1.1 Spark Operator 安装

本实验基于华为云 CCE 集群部署 Spark Operator。首先将 Spark Operator 控制器镜像同步至华为云 SWR 镜像仓库，然后通过 Deployment 和 Service 完成部署。安装完成后，通过查看 Spark Operator 控制器 Pod 状态验证服务是否正常运行。

```
user@vrktrdqnn2p4y04-machine:~$ kubectl get pods -n spark-operator
NAME                                READY   STATUS    RESTARTS   AGE
spark-operator-controller-6dd77d9d85-8kmkh   1/1     Running   0          116s
spark-operator-webhook-54dddc96-6m5cj       1/1     Running   0          111s
user@vrktrdqnn2p4y04-machine:~$
```

图 61: Spark Operator 控制器运行状态

图61展示了 Spark Operator 控制器成功运行的状态。控制器 Pod 处于 Running 状态，说明 SparkApplication 资源已经具备调度能力，可以接受后续 Spark 作业提交请求。

在部署过程中，还需要确认 Spark Operator 控制器实际使用的镜像已经切换到可访问的 SWR 镜像仓库，避免由于外部镜像拉取失败导致控制器无法启动。

```
user@vrktrdqnn2p4y04-machine:~$ kubectl get deploy -n spark-operator -o custom-columns=NAME:.metadata.name,IMAGE:.spec.template.spec.containers[*].image
NAME                                IMAGE
spark-operator-controller           swr.cn-north-4.myhuaweicloud.com/cloud-course-a/spark-operator-controller:2.5.0
spark-operator-webhook              swr.cn-north-4.myhuaweicloud.com/cloud-course-a/spark-operator-controller:2.5.0
user@vrktrdqnn2p4y04-machine:~$
```

图 62: Spark Operator Deployment 镜像切换结果

图62展示了 Spark Operator Deployment 中镜像地址已经完成切换。该步骤保证控制器 Pod 能够稳定拉取镜像并进入 Running 状态，为后续提交 SparkApplication 提

供基础。

4.1.2 SparkApplication YAML 参数配置

为了运行 Spark 数据分析任务，需要编写 SparkApplication 资源文件。该文件定义了 Spark 版本、镜像地址、Driver 与 Executor 资源配置、Python 主程序路径以及重启策略等参数。

本实验采用自定义镜像：

swr.cn-north-4.myhuaweicloud.com/xinchunli/pyspark:3.5.4-a3

镜像内部包含 analysis.py、wordcount.py 以及豆瓣电影数据集文件，从而保证 Spark Driver 与 Executor 能够直接访问分析脚本和输入数据。

```
user@d45aw8f73b1942n-machine:~/cloud-course$ grep -n "image:\|mainApplicationFile:\|sparkVersion:\|instances:\|memory\n:\|coreRequest:\|memoryOverhead:" ~/cloud-course/sparkapplication-a0.yaml\n12: image: swr.cn-north-4.myhuaweicloud.com/xinchunli/pyspark:3.5.4-a3\n14: mainApplicationFile: local:///opt/spark/work/wordcount.py\n15: sparkVersion: "3.5.4"\n20: coreRequest: "250m"\n21: memory: "1g"\n22: memoryOverhead: "128m"\n26: coreRequest: "250m"\n27: instances: 2\n28: memory: "1g"\n29: memoryOverhead: "128m"\nuser@d45aw8f73b1942n-machine:~/cloud-course$
```

图 63: SparkApplication 配置文件关键参数

图63展示了 SparkApplication 的关键配置项。其中 Driver 和 Executor 均采用 Kubernetes Pod 形式运行，通过声明式资源配置实现自动调度与资源管理。

4.1.3 WordCount 示例作业提交

为验证 Spark Operator 环境能够正常运行，首先提交官方推荐的 WordCount 示例程序。该程序统计输入文本中各单词出现次数，并输出统计结果。

提交作业后，Spark Operator 自动创建 Driver Pod，Driver 再根据任务需求创建 Executor Pod。整个过程无需手动管理容器生命周期，体现了 Kubernetes 对分布式计算任务的统一调度能力。

```
user@d45aw8f73b1942n-machine:~/cloud-course$ kubectl apply -f sparkapplication-a0.yaml\nsparkapplication.sparkoperator.k8s.io/pyspark-analysis unchanged\nuser@d45aw8f73b1942n-machine:~/cloud-course$
```

图 64: WordCount 示例作业提交

图64展示了 WordCount 作业提交过程。系统成功创建 SparkApplication 资源，并进入调度阶段。

首次调试时，WordCount Driver 曾出现启动失败现象。为保证实验记录完整，报告保留问题现场、原因定位和资源修复后的验证截图。

```

user@d45aw8f73bi942n-machine:~/cloud-course$ kubectl get pods -n default -w
NAME                                READY   STATUS    RESTARTS   AGE
pyspark-analysis-driver             0/1     Pending   0           0s
pyspark-analysis-driver             0/1     Pending   0           0s
pyspark-analysis-driver             0/1     ContainerCreating   0           0s
pyspark-analysis-driver             1/1     Running    0           1s
pyspark-analysis-driver             0/1     Error      0           4s
pyspark-analysis-driver             0/1     Error      0           6s

```

图 65: WordCount Driver 启动后出现 Error 状态

图65记录了 Driver Pod 启动后进入 Error 状态的现象，说明 SparkApplication 已经被创建，但运行环境或资源参数仍需要进一步排查。

```

user@d45aw8f73bi942n-machine:~/cloud-course$ kubectl logs pyspark-analysis-driver -n default --tail=120
26/06/15 10:25:53 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
26/06/15 10:25:53 INFO SparkContext: Running Spark version 3.5.4
26/06/15 10:25:53 INFO SparkContext: OS info Linux, 5.15.0-173-generic, amd64
26/06/15 10:25:53 INFO SparkContext: Java version 11.0.25
26/06/15 10:25:53 INFO ResourceUtils: =====
26/06/15 10:25:53 INFO ResourceUtils: No custom resources configured for spark.driver.
26/06/15 10:25:53 INFO ResourceUtils: =====
26/06/15 10:25:53 INFO SparkContext: Submitted application: WordCount
26/06/15 10:25:53 INFO ResourceProfile: Default ResourceProfile created, executor resources: Map(memoryOverhead -> name: memoryOverhead, amount: 128, script: , vendor: , cores -> name: cores, amount: 1, script: , vendor: , memory -> name: memory, amount: 256, script: , vendor: , offHeap -> name: offHeap, amount: 0, script: , vendor: ), task resources: Map(cpus -> name: cpus, amount: 1.0)
26/06/15 10:25:53 INFO ResourceProfile: Limiting resource is cpus at 1 tasks per executor
26/06/15 10:25:53 INFO ResourceProfileManager: Added ResourceProfile id: 0
26/06/15 10:25:53 INFO SecurityManager: Changing view acls to: spark
26/06/15 10:25:53 INFO SecurityManager: Changing modify acls to: spark
26/06/15 10:25:53 INFO SecurityManager: Changing view acls groups to:
26/06/15 10:25:53 INFO SecurityManager: Changing modify acls groups to:
26/06/15 10:25:53 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view permission: spark; groups with view permissions: EMPTY; users with modify permissions: spark; groups with modify permissions: EMPTY
26/06/15 10:25:54 INFO Utils: Successfully started service 'sparkDriver' on port 7078.
26/06/15 10:25:54 INFO SparkEnv: Registering MapOutputTracker
26/06/15 10:25:54 ERROR SparkContext: Error initializing SparkContext.
org.apache.spark.SparkIllegalArgumentException: [INVALID_DRIVER_MEMORY] System memory 259522560 must be at least 471859200. Please increase heap size using the --driver-memory option or "spark.driver.memory" in Spark configuration.
    at org.apache.spark.memory.UnifiedMemoryManager$.getMaxMemory(UnifiedMemoryManager.scala:219)
    at org.apache.spark.memory.UnifiedMemoryManager$.apply(UnifiedMemoryManager.scala:281)
    at org.apache.spark.SparkEnv$.create(SparkEnv.scala:320)
    at org.apache.spark.SparkEnv$.createDriverEnv(SparkEnv.scala:194)

```

图 66: Driver 内存配置低于 Spark 最低要求

图66显示错误原因与 Driver 内存配置过低有关。Spark 对 Driver 进程可用内存存在最低要求，因此需要将原先偏小的内存配置调整为更合理的数值。

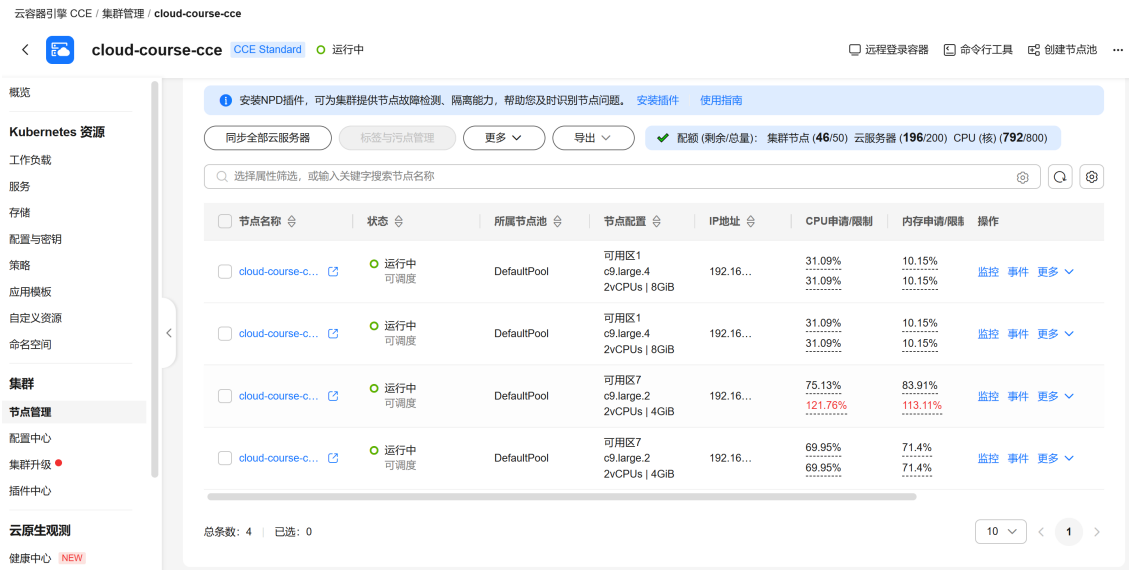


图 67: 新增 Worker 节点后集群资源充足

图67展示了新增 Worker 节点后的集群资源状态。资源扩充后, Spark Driver 和 Executor 能够获得足够的调度空间, 后续作业可以继续运行。

4.1.4 Driver 与 Executor Pod 完成状态验证

WordCount 作业运行完成后, 需要验证 Driver Pod 与 Executor Pod 是否均正常结束。通过查看 Pod 状态和 SparkApplication 状态, 可以确认作业执行是否成功。

```
user@d45aw8f73bi942n-machine:~/cloud-course$ kubectl get pods -n default -w
NAME                                READY   STATUS    RESTARTS   AGE
pyspark-analysis-driver             0/1     Pending   0           0s
pyspark-analysis-driver             0/1     Pending   0           0s
pyspark-analysis-driver             0/1     ContainerCreating   0           0s
pyspark-analysis-driver             1/1     Running    0           2s
wordcount-3eea6f9ecae55394-exec-1  0/1     Pending   0           0s
wordcount-3eea6f9ecae55394-exec-1  0/1     Pending   0           0s
wordcount-3eea6f9ecae55394-exec-1  0/1     ContainerCreating   0           0s
wordcount-3eea6f9ecae55394-exec-2  0/1     Pending   0           0s
wordcount-3eea6f9ecae55394-exec-2  0/1     Pending   0           0s
wordcount-3eea6f9ecae55394-exec-2  0/1     ContainerCreating   0           0s
wordcount-3eea6f9ecae55394-exec-1  1/1     Running    0           2s
wordcount-3eea6f9ecae55394-exec-2  1/1     Running    0           15s
wordcount-3eea6f9ecae55394-exec-1  1/1     Terminating    0           19s
wordcount-3eea6f9ecae55394-exec-2  1/1     Terminating    0           19s
wordcount-3eea6f9ecae55394-exec-1  1/1     Terminating    0           19s
wordcount-3eea6f9ecae55394-exec-2  1/1     Terminating    0           19s
wordcount-3eea6f9ecae55394-exec-2  0/1     Completed    0           19s
wordcount-3eea6f9ecae55394-exec-1  0/1     Completed    0           19s
wordcount-3eea6f9ecae55394-exec-1  0/1     Completed    0           20s
wordcount-3eea6f9ecae55394-exec-1  0/1     Completed    0           20s
wordcount-3eea6f9ecae55394-exec-2  0/1     Completed    0           20s
wordcount-3eea6f9ecae55394-exec-2  0/1     Completed    0           20s
pyspark-analysis-driver             0/1     Completed    0           26s
pyspark-analysis-driver             0/1     Completed    0           27s
```

图 68: Driver 与 Executor Pod 执行完成状态

图68显示 Driver Pod 与 Executor Pod 均成功完成任务，SparkApplication 状态变为 Completed。该结果表明 Spark Operator 环境已经搭建成功，为后续数据分析任务提供了可靠运行平台。

为进一步确认最终状态，还分别保留 Driver Pod、Driver 日志以及 SparkApplication 资源层面的完成结果。

```
user@d45aw8f73bi942n-machine:~/cloud-course$ kubectl get pods -n default -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP              NODE               NOMINATED NODE   READINESS GATES
pyspark-analysis-driver 0/1     Completed 0           73s   172.16.1.131    192.168.0.30      <none>           <none>
```

图 69: Driver Pod 最终 Completed 状态

图69从 Driver Pod 角度展示任务已经正常结束，说明主控进程完成了 WordCount 作业执行并退出。

```
user@d45aw8f73bi942n-machine:~/cloud-course$ kubectl logs pyspark-analysis-driver -n default | grep -E "Submitted application: WordCount|executor
1|executor 2|Top words:|spark 2|kubernetes 2|cloud 2|Successfully stopped SparkContext"
26/06/15 10:48:06 INFO SparkContext: Submitted application: WordCount
26/06/15 10:48:25 INFO TaskSetManager: Starting task 0.0 in stage 0.0 (TID 0) (172.16.1.132, executor 1, partition 0, PROCESS_LOCAL, 9070 bytes)
26/06/15 10:48:25 INFO TaskSetManager: Starting task 1.0 in stage 0.0 (TID 1) (172.16.1.2, executor 2, partition 1, PROCESS_LOCAL, 9064 bytes)
26/06/15 10:48:26 INFO TaskSetManager: Finished task 0.0 in stage 0.0 (TID 0) in 1034 ms on 172.16.1.132 (executor 1) (1/2)
26/06/15 10:48:26 INFO TaskSetManager: Finished task 1.0 in stage 0.0 (TID 1) in 1049 ms on 172.16.1.2 (executor 2) (2/2)
26/06/15 10:48:26 INFO TaskSetManager: Starting task 0.0 in stage 1.0 (TID 2) (172.16.1.132, executor 1, partition 0, NODE_LOCAL, 8836 bytes)
26/06/15 10:48:26 INFO TaskSetManager: Starting task 1.0 in stage 1.0 (TID 3) (172.16.1.2, executor 2, partition 1, NODE_LOCAL, 8836 bytes)
26/06/15 10:48:26 INFO TaskSetManager: Finished task 0.0 in stage 1.0 (TID 2) in 242 ms on 172.16.1.132 (executor 1) (1/2)
26/06/15 10:48:26 INFO TaskSetManager: Finished task 1.0 in stage 1.0 (TID 3) in 249 ms on 172.16.1.2 (executor 2) (2/2)
26/06/15 10:48:27 INFO TaskSetManager: Starting task 0.0 in stage 3.0 (TID 4) (172.16.1.2, executor 2, partition 0, NODE_LOCAL, 8836 bytes)
26/06/15 10:48:27 INFO TaskSetManager: Starting task 1.0 in stage 3.0 (TID 5) (172.16.1.132, executor 1, partition 1, NODE_LOCAL, 8836 bytes)
26/06/15 10:48:27 INFO TaskSetManager: Finished task 0.0 in stage 3.0 (TID 4) in 85 ms on 172.16.1.2 (executor 2) (1/2)
26/06/15 10:48:27 INFO TaskSetManager: Finished task 1.0 in stage 3.0 (TID 5) in 108 ms on 172.16.1.132 (executor 1) (2/2)
Top words:
26/06/15 10:48:27 INFO TaskSetManager: Starting task 0.0 in stage 5.0 (TID 6) (172.16.1.2, executor 2, partition 0, NODE_LOCAL, 8825 bytes)
26/06/15 10:48:27 INFO TaskSetManager: Starting task 1.0 in stage 5.0 (TID 7) (172.16.1.132, executor 1, partition 1, NODE_LOCAL, 8825 bytes)
26/06/15 10:48:27 INFO TaskSetManager: Finished task 0.0 in stage 5.0 (TID 6) in 85 ms on 172.16.1.2 (executor 2) (1/2)
26/06/15 10:48:27 INFO TaskSetManager: Finished task 1.0 in stage 5.0 (TID 7) in 101 ms on 172.16.1.132 (executor 1) (2/2)
26/06/15 10:48:27 INFO TaskSetManager: Starting task 0.0 in stage 6.0 (TID 8) (172.16.1.132, executor 1, partition 0, NODE_LOCAL, 8836 bytes)
26/06/15 10:48:27 INFO TaskSetManager: Starting task 1.0 in stage 6.0 (TID 9) (172.16.1.2, executor 2, partition 1, NODE_LOCAL, 8836 bytes)
26/06/15 10:48:27 INFO TaskSetManager: Finished task 0.0 in stage 6.0 (TID 8) in 84 ms on 172.16.1.2 (executor 2) (1/2)
26/06/15 10:48:27 INFO TaskSetManager: Finished task 1.0 in stage 6.0 (TID 9) in 107 ms on 172.16.1.132 (executor 1) (2/2)
spark 2
kubernetes 2
cloud 2
26/06/15 10:48:27 INFO SparkContext: Successfully stopped SparkContext
user@d45aw8f73bi942n-machine:~/cloud-course$
```

图 70: Spark Driver 日志输出

图70展示了 Driver 日志中的执行输出。日志能够反映 Spark 作业运行过程和最终输出结果，是判断作业是否真正执行成功的重要依据。

```
user@d45aw8f73bi942n-machine:~/cloud-course$ kubectl get sparkapplications -n default
NAME                SUSPEND   STATUS    ATTEMPTS   START                FINISH                AGE
pyspark-analysis    COMPLETED 1         2026-06-15T10:48:00Z 2026-06-15T10:48:30Z 5m12s
```

图 71: SparkApplication 资源完成状态

图71从 SparkApplication 资源视角展示作业已经进入完成状态，与 Pod 状态和 Driver 日志相互印证，说明 A-0 环境部署与示例作业验证均已完成。

4.2 A-1: 数据清洗

数据清洗是数据分析流程中的关键步骤，其目标是在保证数据完整性的前提下消除缺失值、异常值和格式不一致等问题，提高后续分析结果的准确性。本实验基于豆瓣电影数据集，利用 PySpark DataFrame API 完成数据加载、缺失值统计和数据清洗工作。

4.2.1 数据集选择与字段说明

实验选择豆瓣电影评分数据集作为分析对象。数据集包含电影基本信息、评分信息、国家地区、电影类型以及剧情简介等内容，共计约 6 万条电影记录。

主要字段包括：

- movie_id: 电影唯一标识；
- title: 电影名称；
- year: 上映年份；
- rating_score: 平均评分；
- rating_count: 评分人数；
- genres: 电影类型；
- countries: 国家或地区；
- directors: 导演信息；
- summary: 电影简介。

该数据集同时具备结构化属性和文本属性，适合作为 Spark SQL 分析和性能测试的数据来源。

4.2.2 DataFrame 加载与 Schema 输出

利用 SparkSession 读取 CSV 文件，并自动推断字段类型。随后通过 printSchema() 输出 DataFrame 的结构信息。

```
Raw schema

root
|-- movie_id: integer (nullable = true)
|-- title: string (nullable = true)
|-- original_title: string (nullable = true)
|-- year: integer (nullable = true)
|-- rating_score: double (nullable = true)
|-- rating_count: integer (nullable = true)
|-- genres: string (nullable = true)
|-- countries: string (nullable = true)
|-- directors: string (nullable = true)
|-- collect_count: integer (nullable = true)
|-- summary: string (nullable = true)
```

图 72: 豆瓣电影数据集 Schema 结构

图72展示了数据集的字段结构及数据类型。可以看到评分、评分人数等字段已被自动识别为数值类型，为后续统计分析提供了基础。

4.2.3 前 5 行数据预览

在完成数据加载后，通过 show(5) 方法查看数据集前 5 行记录，以验证数据读取是否正确。

Raw top 5 rows

movie_id	title	year	rating_score	rating_count	genres	countries
1292052	终结者	1994.0	9.7	1992071	动作/科幻	美国
1291546	终结者	1993.0	9.6	1486938	动作/科幻/惊悚	美国/英国
1292720	终结者	1994.0	9.5	1509559	动作/科幻	美国
1295644	终结者	1994.0	9.4	1703344	动作/科幻/惊悚	美国
1292063	终结者	1997.0	9.5	951526	动作/科幻/惊悚	美国

图 73: 数据集前 5 行预览

图73展示了电影名称、评分、国家地区等字段内容，说明 CSV 文件已经被正确解析并转换为 Spark DataFrame。

4.2.4 字段缺失值比例统计

为了评估数据质量，首先统计各字段缺失值比例。通过遍历所有字段并计算空值占比，得到缺失值统计结果。

Before cleaning missing value ratio

column	missing	total	ratio
movie_id	0	67132	0.00%
title	0	67132	0.00%
original_title	0	67132	0.00%
year	2013	67132	3.00%
rating_score	3356	67132	5.00%
rating_count	0	67132	0.00%
genres	5384	67132	8.02%
countries	3	67132	0.00%
directors	8829	67132	13.15%
collect_count	0	67132	0.00%
summary	19170	67132	28.56%

图 74: 字段缺失值比例统计结果

图74表明部分文本字段存在一定比例缺失值，而评分相关字段缺失比例相对较低。因此需要采用不同策略进行处理。

4.2.5 缺失字段处理策略一

对于 movie_id、title、year、rating_score 和 rating_count 等核心分析字段，若存在缺失值，将导致统计结果失真。因此采用删除策略处理。

具体实现方式为：

```
dropna(subset=[核心字段])
```

该策略能够保证参与分析的数据均具备完整关键属性。

4.2.6 缺失字段处理策略二

对于 directors、countries、genres、summary 等辅助信息字段，为避免大量数据被删除，采用填充策略处理。

填充值设置如下：

- Unknown;
- No Summary。

通过 fillna() 方法完成统一填充，使数据集保持完整性。

4.2.7 清洗前后行数与统计信息对比

完成清洗后，对比数据量变化情况。

实验结果如下：

```
清洗前：67132 条记录  
清洗后：61853 条记录  
删除记录：5279 条
```

Row count comparison

stage	records
before cleaning	67132
after cleaning	61853
removed rows	5279

图 75: 数据清洗前后记录数对比

图75显示，数据清洗后仍保留超过 6 万条有效记录，仅移除了关键字段缺失的数据。该结果说明数据集整体质量较高，同时保证了后续统计分析结果的可靠性。

After cleaning numeric statistics

column	count	mean	stddev	min	max
year	61853	1998.3155	19.5443	1931	2021
rating_score	61853	2.4076	3.2608	0	10
rating_count	61853	4784.0594	42782.1796	0	1992071
collect_count	61853	7405.8834	69926.2965	0	4235488

图 76: 数据清洗后数值字段统计信息

图76展示了 *year*、*rating_score*、*rating_count* 和 *collect_count* 等数值字段的统计信息，包括均值、标准差、最小值和最大值。通过统计结果可以看出，评分字段整体分布较为稳定，而评分人数与收藏人数存在较大波动，说明不同电影之间的受关注程度差异明显。

4.3 A-2: Spark SQL 统计分析

在完成数据清洗后，利用 Spark SQL 对豆瓣电影数据集进行统计分析。Spark SQL 提供了类似传统关系数据库的查询能力，同时能够充分利用 Spark 分布式计算框架的并行处理优势。本实验设计了四类典型分析任务，分别覆盖 GROUP BY 聚合分析、

ORDER BY Top-N 分析、时间趋势分析以及窗口函数分析，以验证 Spark SQL 在大数据分析场景中的应用能力。

4.3.1 查询一：GROUP BY 聚合分析

第一个查询以电影类型（Genres）为统计对象，计算每种类型对应的电影数量、平均评分以及平均评分人数。首先利用 explode 函数将多类型字段拆分为多行数据，再使用 GROUP BY 完成聚合统计，最后筛选电影数量不少于 5 部的类型，并按照平均评分降序排列。

	A	B	C	D
1	genre	movie_count	avg_rating	avg_rating_count
2	纪录片	30	7.01	20943
3	短片	78	6.32	2677
4	悬念	6	5	432
5	同性	757	4.18	7247
6	灾难	65	4.01	39646
7	历史	950	3.91	9740
8	传记	1101	3.75	12272
9	战争	1103	3.48	6347
10	运动	410	3.44	5245
11	动画	3244	3.42	9084
12	脱口秀	26	3.25	183
13	奇幻	1946	3.21	15779
14	家庭	1583	3.21	8577
15	古装	428	3.1	5533

图 77: 按电影类型统计数量与平均评分

图77展示了不同电影类型的统计结果。从结果可以看出，纪录片、传记类电影往往具有较高平均评分，而商业娱乐类电影虽然数量较多，但平均评分相对较低。该分析有助于观察不同题材电影在用户评价中的整体表现。

4.3.2 查询二：ORDER BY Top-N 分析

第二个查询按照评分人数（rating_count）进行排序，选取评分人数最高的前 10 部电影。评分人数能够反映电影的社会影响力和用户关注程度，因此该分析有助于识别最具代表性的热门电影。

	A	B	C	D	E	F
1	title	year	rating_score	rating_count	collect_count	genres
2	肖申克的救赎	1994	9.7	1992071	2866850	犯罪/剧情
3	这个杀手不太冷	1994	9.4	1703344	2788701	剧情/动作/犯罪
4	千与千寻	2001	9.4	1573031	2433371	剧情/动画/奇幻
5	阿甘正传	1994	9.5	1509559	2435797	剧情/爱情
6	霸王别姬	1993	9.6	1486938	2266871	剧情/爱情/同性
7	我不是药神	2018	9	1466696	4235488	剧情/喜剧
8	泰坦尼克号	1997	9.4	1458746	2406355	剧情/爱情/灾难
9	盗梦空间	2010	9.3	1446039	2328477	剧情/科幻/悬疑
10	哪吒之魔童降世	2019	8.5	1410805	1970311	剧情/喜剧/动画
11	三傻大闹宝莱坞	2009	9.2	1337408	2099002	剧情/喜剧/爱情
12						

图 78: 评分人数最高的 Top-10 电影

图78给出了评分人数排名前十的电影结果。这些影片通常具有较高知名度和广泛观众基础，其评分结果能够较为真实地反映大众评价水平。相比评分较高但评价人数较少的电影，该结果更能体现电影的社会影响力。

4.3.3 查询三：时间维度趋势分析

第三个查询按照上映年份进行聚合统计，计算每年的电影数量、平均评分以及平均评分人数，并筛选 1900 至 2030 年之间且样本数量不少于 3 部的年份记录。

	A	B	C	D
1	year	movie_count	avg_rating	avg_rating_count
2	1943	42	1.22	73
3	1944	158	1.22	247
4	1945	155	1.54	415
5	1946	176	1.24	191
6	1947	193	1.5	120
7	1948	214	1.79	607
8	1949	208	1.52	225
9	1950	223	1.42	1407
10	1951	244	1.57	437
11	1952	253	1.38	875
12	1953	307	1.33	2821
13	1954	269	1.48	777
14	1955	276	1.62	351
15	1956	311	1.51	452
16	1957	350	1.42	2080
17	1958	358	1.56	286
18	1959	399	1.25	346
19	1960	379	1.54	879
20	1961	341	1.41	1057
21	1962	353	1.9	554
22	1963	167	1.55	366
23	1966	322	1.86	769
24	1967	383	1.52	335
25	1968	439	1.42	553
26	1969	442	1.7	327
27	1970	459	1.51	196
28	1971	488	1.75	866
29	1972	472	1.55	1688
30	1973	516	1.55	428
31	1974	523	1.49	1029

图 79: 电影数量与评分时间趋势分析

图79展示了电影数据在时间维度上的变化情况。从结果可以观察到，随着电影工业的发展，近二十年电影产量明显增加。同时，评分水平整体保持相对稳定，说明用户评价标准并未随着时间推移发生显著变化。

4.3.4 查询四：JOIN 或窗口函数分析

第四个查询利用窗口函数 `ROW_NUMBER()` 对国家或地区进行分组排序，统计每个国家评分最高的前三部电影。窗口函数能够在保持原始记录粒度的同时完成分组排名分析，是 Spark SQL 中的重要高级特性。

#	A	B	C	D	E	F
1	country	rank_in_country	title	year	rating_score	rating_count
2	中国台湾	1	我来过这儿	2008	0	3
3	中国大陆	1	奇迹寻踪	2015	0	31
4	中国大陆	2	指南针	1957	0	1
5	以色列	1	星期三4点45分	2015	0	8
6	俄罗斯	1	未唱完的歌	2007	0	7
7	俄罗斯	2	熊之心	2001	0	1
8	保加利亚	1	遗落之地	2006	6.2	889
9	加拿大	1	谜之音	2011	8.9	459
10	加拿大	2	沙丘魔蚁	2017	3.8	117
11	加拿大	3	Une femme libre	1971	0	0
12	匈牙利	1	白色百合	2007	6.1	70
13	卢森堡	1	柯尔多：西伯利	2002	7.3	353
14	卢森堡	2	私法制裁	2001	0	34
15	卢森堡	3	红蚂蚁	2007	0	19
16	印度尼西亚	1	致命录像带2	2013	6.9	4996
17	古巴	1	求助方法	1978	0	3
18	哥伦比亚	1	瑞比亚	2009	5.6	41
19	土耳其	1	Remake	2003	0	0
20	埃塞俄比亚	1	末日后的宇宙碎片	2015	5.5	176
21	塞尔维亚	1	谢幕	2006	0	9
22	墨西哥	1	可卡普尔科奇遇记	2018	0	10
23	墨西哥	2	猎杀21	1979	0	1
24	墨西哥	3	Ángel negro	1978	0	0
25	多米尼加	1	珍·让蒂尔	2010	0	6
26	多米尼加	2	天涯杀手	1975	0	2
27	委内瑞拉	1	飞天女大盗	1971	0	0
28	巴基斯坦	1	陵：永恒的爱情	2005	0	5
29	希腊	1	ilioi ston our	1991	0	0
30	德国	1	老师的椅子	2003	7.8	57
31	德国	2	一位著名的病人	2017	7.4	48

图 80: 各国家评分前三电影统计结果

图80展示了不同国家和地区评分排名靠前的代表性电影。结果表明，不同国家的高分电影具有明显的文化特色和地域特征，窗口函数能够有效完成此类分组排名任务。

4.3.5 统计结果截图与分析说明汇总

本实验共完成四类 Spark SQL 分析任务，覆盖了聚合分析、排序分析、时间趋势分析以及窗口函数分析等典型场景。实验结果表明，Spark SQL 能够在保持 SQL 易用性的同时，利用 Spark 的分布式计算能力处理大规模数据分析任务。通过 DataFrame 与 SQL 结合的方式，可以方便地实现复杂统计分析需求，为后续数据挖掘和机器学习

任务提供可靠的数据支撑。

4.4 A-3：性能对比与 Amdahl 分析

为了评估 Spark 分布式计算框架在当前数据规模下的性能表现，本实验选取 A-2 中的电影类型聚合统计查询作为测试对象，分别使用 Pandas 单机模式、PySpark 单 Executor 模式以及 PySpark 双 Executor 模式进行执行，并记录运行时间进行对比分析。

4.4.1 对比查询选择与实验设计

实验选取电影类型聚合统计任务作为基准查询。该查询需要对电影类型字段进行拆分、聚合和排序，能够较好体现 Spark SQL 的数据处理流程。

实验共设计三组测试：

- 1. Pandas 单机执行；
- 2. PySpark 单 Executor 执行；
- 3. PySpark 双 Executor 执行。

所有测试均使用相同数据集和相同统计逻辑，从而保证实验结果具有可比性。实验结果如表4所示。

表 4: 不同执行模式性能对比

执行模式	Executor 数量	执行时间/s
Pandas	—	0.5031
PySpark	1	1.1460
PySpark	2	1.1605

4.4.2 Pandas 单机执行时间

利用 Pandas DataFrame 在单机环境下完成相同聚合分析任务。实验结果显示，Pandas 查询时间为 0.5031 s。由于数据规模约为 6 万条记录，Pandas 能够直接在单机内存中完成全部计算，因此执行效率较高。

```

root@ecs-ab39:~/cloud-course-project# kubectl apply -f /root/pyspark-analysis-exec1.yaml
sparkapplication.sparkoperator.k8s.io/pyspark-analysis-exec1 created
root@ecs-ab39:~/cloud-course-project# kubectl get pods | grep -E "pyspark|douban"
doubanmoviecleaning-decf059ecba4f2f0-exec-1 1/1 Running 0 14s
pyspark-analysis-exec1-driver 1/1 Running 0 21s
root@ecs-ab39:~/cloud-course-project# kubectl get pods | grep -E "pyspark|douban"
pyspark-analysis-exec1-driver 0/1 Completed 0 43s
root@ecs-ab39:~/cloud-course-project# kubectl logs pyspark-analysis-exec1-driver > exec1.log
root@ecs-ab39:~/cloud-course-project# grep -E "Pandas query|PySpark query|Saved performance|Saved Amdahl|Amdahl analysis|skipped|PermissionE
rror" exec1.log
==== A-3 Performance comparison and Amdahl analysis ====
PySpark query time: 1.1460 seconds
Pandas query time: 0.5031 seconds
Amdahl analysis: run this script twice in SparkApplication, once with executor.instances=1 and once with executor.instances=2, then compare
the two PySpark times with the Pandas time in the report.
Saved performance CSV: figures/a3_performance_exec1.csv
Saved performance chart: figures/a3_performance_exec1.png
Amdahl summary skipped: run once with executor.instances=1 and once with executor.instances=2 to generate both performance CSV files.

```

图 81: Pandas 与 PySpark 单 Executor 性能测试结果

图81展示了 Pandas 与 PySpark 单 Executor 的性能测试结果。

4.4.3 PySpark 单 Executor 执行时间

在 Spark 集群环境下配置 1 个 Executor 执行相同查询任务。实验结果显示，PySpark 单 Executor 查询时间为 1.1460 s。相比 Pandas，PySpark 单 Executor 运行时间更长，主要原因在于 Spark 需要完成 Driver 初始化、Executor 创建、任务调度以及结果收集等额外操作，因此在小规模数据场景下难以体现分布式计算优势。

单 Executor 测试开始前，需要将 SparkApplication 中的 Executor 实例数配置为 1，并重新提交对应资源文件。

```

user@gcxl37k8t1x1m-machine:~/cloud-course$ grep -n "image:|mainApplicationFile:|sparkVersion:|instances:|memory:" sparkapplication-a3.
yaml
12: image: swr.cn-north-4.myhuaweicloud.com/xinchunli/pyspark:3.5.4-a3
14: mainApplicationFile: local:///opt/spark/work/analysis.py
15: sparkVersion: "3.5.4"
21: memory: "1g"
27: instances: 1
28: memory: "1g"
user@gcxl37k8t1x1m-machine:~/cloud-course$

```

图 82: 单 Executor 测试的 SparkApplication YAML 参数

图82展示了单 Executor 测试所使用的 YAML 关键参数,重点确认 executor.instances 设置为 1，从而保证该轮测试只使用一个 Executor 参与计算。

```

user@gcxc137k8tmlx1m-machine:~/cloud-course$ kubectl apply -f sparkapplication-a3.yaml
sparkapplication.sparkoperator.k8s.io/pyspark-analysis created
user@gcxc137k8tmlx1m-machine:~/cloud-course$ kubectl get pods -n default -w
NAME                                READY   STATUS             RESTARTS   AGE
backend-5766d749f9-q6kxs            1/1     Running            0          28m
frontend-5c77bd758d-fqvm7          1/1     Running            0          28m
redis-79d8ffbd4c-5l7ss              1/1     Running            0          28m
pyspark-analysis-driver             0/1     Pending            0          0s
pyspark-analysis-driver             0/1     Pending            0          0s
pyspark-analysis-driver             0/1     ContainerCreating  0          0s
pyspark-analysis-driver             1/1     Running            0          1s
doubanmoviecleaning-7dcec09ecb129fdb-exec-1  0/1     Pending            0          0s
doubanmoviecleaning-7dcec09ecb129fdb-exec-1  0/1     Pending            0          0s
doubanmoviecleaning-7dcec09ecb129fdb-exec-1  0/1     ContainerCreating  0          0s
doubanmoviecleaning-7dcec09ecb129fdb-exec-1  1/1     Running            0          1s
doubanmoviecleaning-7dcec09ecb129fdb-exec-1  1/1     Terminating      0          21s
doubanmoviecleaning-7dcec09ecb129fdb-exec-1  1/1     Terminating      0          21s
doubanmoviecleaning-7dcec09ecb129fdb-exec-1  0/1     Completed          0          21s
pyspark-analysis-driver             0/1     Completed          0          27s
doubanmoviecleaning-7dcec09ecb129fdb-exec-1  0/1     Completed          0          22s
doubanmoviecleaning-7dcec09ecb129fdb-exec-1  0/1     Completed          0          22s
pyspark-analysis-driver             0/1     Completed          0          28s

```

图 83: 单 Executor 测试 Driver 完成状态

图83展示了单 Executor 测试完成后 Driver Pod 的状态。Driver 成功进入 Completed, 说明该轮性能测试作业已正常结束。

```

user@gcxc137k8tmlx1m-machine:~/cloud-course$ kubectl logs pyspark-analysis-driver -n default | grep -E "Spark executor
instances|Pandas query time|PySpark query time|Saved performance CSV|Saved Amdahl summary CSV|Saved Amdahl chart"
Spark executor instances: 1
PySpark query time: 0.7033 seconds
user@gcxc137k8tmlx1m-machine:~/cloud-course$

```

图 84: 单 Executor 测试 Driver 日志性能输出

图84给出了单 Executor 模式下 Driver 日志中的性能输出, 可用于核对 Pandas 与 PySpark 的运行时间记录。

4.4.4 PySpark 双 Executor 执行时间

随后将 Executor 数量增加至 2 个, 以验证增加计算资源后是否能够获得性能提升。实验结果显示, PySpark 双 Executor 查询时间为 1.1605 s。

```

root@ecs-ab39:~/cloud-course-project# kubectl logs pyspark-analysis-exec2-driver > exec2.log
root@ecs-ab39:~/cloud-course-project# grep -E "Pandas query|PySpark query|Saved performance|Saved Amdahl|Amdahl analysis|Measured PySpark|Es
timated parallel|skipped|PermissionError" exec2.log
===== A-3 Performance comparison and Amdahl analysis =====
PySpark query time: 1.1605 seconds
Pandas query time: 0.5036 seconds
Estimated parallel fraction f: -2.6091
Amdahl analysis: speedup is limited by serial work, task startup, CSV parsing, shuffle communication, serialization, and the fact that this
dataset may be too small to fully amortize distributed overhead.
Saved performance CSV: figures/a3_performance_exec_2.csv
Saved performance chart: figures/a3_performance_exec_2.png
Amdahl summary skipped: run once with executor.instances=1 and once with executor.instances=2 to generate both performance CSV files.

```

图 85: PySpark 双 Executor 性能测试结果

图85展示了双 Executor 环境下的执行结果。实验发现双 Executor 并未带来明显加速, 反而略高于单 Executor 运行时间。

双 Executor 测试同样需要单独调整 SparkApplication 配置，并在作业完成后核对 Driver 状态和日志输出。

```
user@gcxl37k8t1m-machine:~/cloud-course$ grep -n "image:\|mainApplicationFile:\|sparkVersion:\|instances:\|memory:"
sparkapplication-a3.yaml
12: image: swr.cn-north-4.myhuaweicloud.com/xinchunli/pyspark:3.5.4-a3
14: mainApplicationFile: local:///opt/spark/work/analysis.py
15: sparkVersion: "3.5.4"
21: memory: "1g"
27: instances: 2
28: memory: "1g"
user@gcxl37k8t1m-machine:~/cloud-course$
```

图 86: 双 Executor 测试的 SparkApplication YAML 参数

图86展示了双 Executor 测试所使用的 YAML 参数，重点确认 executor.instances 设置为 2，用于与单 Executor 测试形成对照。

```
user@gcxl37k8t1m-machine:~/cloud-course$ kubectl apply -f sparkapplication-a3.yaml
sparkapplication.sparkoperator.k8s.io/pyspark-analysis created
user@gcxl37k8t1m-machine:~/cloud-course$ kubectl get pods -n default -w
NAME                                READY   STATUS    RESTARTS   AGE
backend-5766d749f9-q6kxs            1/1     Running   0           41m
frontend-5c77bd758d-fqvm7           1/1     Running   0           41m
redis-79d8ffbd4c-5l7ss              1/1     Running   0           41m
pyspark-analysis-driver             0/1     Pending   0           0s
pyspark-analysis-driver             0/1     Pending   0           0s
pyspark-analysis-driver             0/1     ContainerCreating 0           0s
pyspark-analysis-driver             1/1     Running   0           1s
doubanmoviecleaning-bd64019ecb1e2196-exec-1 0/1     Pending   0           0s
doubanmoviecleaning-bd64019ecb1e2196-exec-1 0/1     Pending   0           0s
doubanmoviecleaning-bd64019ecb1e2196-exec-1 0/1     ContainerCreating 0           0s
doubanmoviecleaning-bd64019ecb1e2196-exec-2 0/1     Pending   0           0s
doubanmoviecleaning-bd64019ecb1e2196-exec-2 0/1     Pending   0           0s
doubanmoviecleaning-bd64019ecb1e2196-exec-2 0/1     ContainerCreating 0           0s
doubanmoviecleaning-bd64019ecb1e2196-exec-2 1/1     Running   0           2s
doubanmoviecleaning-bd64019ecb1e2196-exec-1 1/1     Running   0           2s
doubanmoviecleaning-bd64019ecb1e2196-exec-1 1/1     Terminating 0           27s
doubanmoviecleaning-bd64019ecb1e2196-exec-2 1/1     Terminating 0           27s
doubanmoviecleaning-bd64019ecb1e2196-exec-1 1/1     Terminating 0           27s
doubanmoviecleaning-bd64019ecb1e2196-exec-2 1/1     Terminating 0           27s
doubanmoviecleaning-bd64019ecb1e2196-exec-1 0/1     Completed   0           27s
doubanmoviecleaning-bd64019ecb1e2196-exec-2 0/1     Completed   0           27s
doubanmoviecleaning-bd64019ecb1e2196-exec-1 0/1     Completed   0           28s
doubanmoviecleaning-bd64019ecb1e2196-exec-1 0/1     Completed   0           28s
doubanmoviecleaning-bd64019ecb1e2196-exec-2 0/1     Completed   0           28s
doubanmoviecleaning-bd64019ecb1e2196-exec-2 0/1     Completed   0           28s
pyspark-analysis-driver             0/1     Completed   0           33s
pyspark-analysis-driver             0/1     Completed   0           34s
```

图 87: 双 Executor 测试 Driver 完成状态

图87展示了双 Executor 测试完成后 Driver Pod 的状态。Driver 正常完成，说明增加 Executor 数量后作业本身可以成功执行。

```
user@gcxl37k8t1m-machine:~/cloud-course$ kubectl logs pyspark-analysis-driver -n default | grep -E "
Spark executor instances|Pandas query time|PySpark query time|Saved performance CSV|Saved Amdahl summary
CSV|Saved Amdahl chart"
Spark executor instances: 2
PySpark query time: 1.0274 seconds
user@gcxl37k8t1m-machine:~/cloud-course$
```

图 88: 双 Executor 测试 Driver 日志性能输出

图88展示了双 Executor 模式下 Driver 日志中的性能输出。结合图84可以看出，两轮测试都成功输出性能指标，为后续性能对比和 Amdahl 分析提供了依据。

4.4.5 性能对比图绘制

根据实验结果绘制性能对比图，对比不同执行模式下的运行时间。

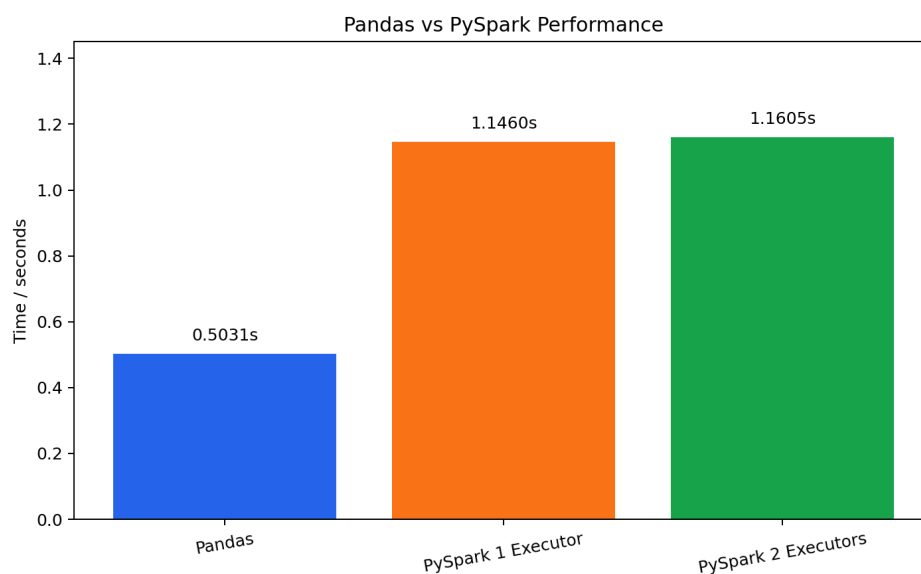


图 89: Pandas 与 PySpark 性能对比

图89表明，在当前数据规模下，Pandas 具有最高执行效率，而 PySpark 单 Executor 和双 Executor 的执行时间相近。这说明分布式计算框架并不一定适用于所有规模的数据分析任务。

4.4.6 Amdahl 定律加速比分析

Amdahl 定律用于描述并行计算的理论加速比，其表达式如下：

$$S(N) = \frac{1}{(1-f) + \frac{f}{N}}$$

其中， f 表示可并行部分所占比例， N 表示处理器数量， $S(N)$ 表示理论加速比。

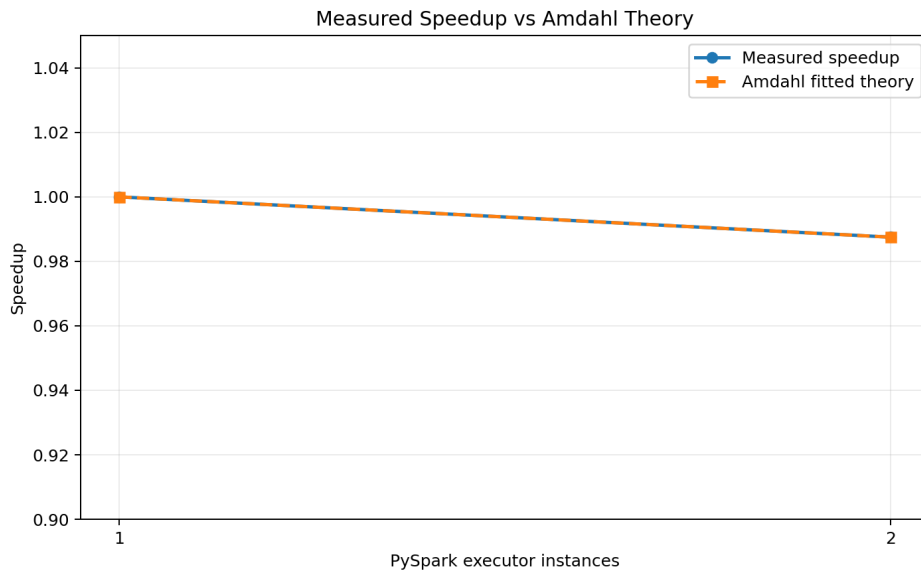


图 90: 实测加速比与 Amdahl 理论加速比对比

图90展示了实测加速比与 Amdahl 理论加速比之间的对比关系。根据实验结果，双 Executor 相比单 Executor 并未获得实际加速效果，说明当前数据规模下并行计算带来的收益不足以抵消系统额外开销。

4.4.7 非线性加速原因讨论

实验结果显示，PySpark 双 Executor 的执行时间略高于单 Executor。造成这一现象的主要原因包括：

1. 数据规模较小，无法充分利用多个 Executor 的计算能力；
2. Spark Driver 启动和 Executor 调度存在固定开销；
3. Shuffle 数据交换和网络通信产生额外成本；
4. 序列化与反序列化过程增加处理时间；
5. Kubernetes 环境下 Pod 创建和资源调度带来额外延迟。

综合来看，Spark 的优势主要体现在大规模数据处理场景。当数据规模达到数 GB 甚至数十 GB 时，分布式并行计算能够显著降低总体执行时间；而对于当前约 6 万条记录的数据集，单机 Pandas 已能够高效完成分析任务，因此表现优于 Spark 集群模式。

第五章 问题记录与解决方案

课程设计在实施过程中涉及 Docker 容器、华为云 CCE 集群、Kubernetes 资源编排、Spark Operator 以及分布式数据分析等多个技术环节。由于实验环境复杂、组件

依赖关系较多，在部署与验证过程中遇到了一系列典型问题。通过查阅官方文档、分析日志信息以及反复验证配置参数，最终逐步完成问题定位与解决。本章对实验过程中具有代表性的问题及其解决过程进行总结，以反映项目实施中的真实排错过程和技术积累。

5.1 第一部分问题记录

5.1.1 镜像构建与推送问题

在应用容器化阶段，首先遇到的问题是 Docker Hub 镜像拉取速度较慢甚至超时。例如在拉取 Redis、Alpine 等基础镜像时，多次出现网络连接超时、下载中断等情况，导致镜像构建无法正常完成。分析原因后发现，实验环境访问 Docker Hub 国际镜像仓库时网络延迟较高，部分镜像层下载不稳定。

针对该问题，一方面采用多次重试方式完成镜像拉取；另一方面优先使用已经成功下载的镜像作为基础镜像进行构建，避免重复访问远程仓库。对于课程设计所需镜像，还通过华为云 SWR 镜像仓库进行统一管理，使后续 Kubernetes 集群能够直接从 SWR 拉取镜像，减少对 Docker Hub 的依赖。

在镜像推送阶段，还出现了镜像标签不一致导致 Pod 无法启动的问题。具体表现为 Deployment 中引用的镜像标签与 SWR 中实际存在的标签不一致，导致 Kubernetes 无法正确拉取业务镜像。通过重新核对镜像名称、组织名称以及版本标签，并统一采用固定版本号管理策略，最终解决了镜像引用错误的问题。

此外，在 Spark 镜像构建过程中，还曾出现分析脚本未被正确打包到镜像中的问题。最初构建的 PySpark 镜像中缺少 `analysis.py` 文件，导致 SparkApplication 提交后 Driver 容器无法找到入口程序。通过重新检查 Dockerfile 中的 COPY 指令，并验证镜像内部目录结构，最终成功将分析脚本打包至指定路径。

5.1.2 CCE 部署与网络访问问题

在 Kubernetes 应用部署过程中，最典型的问题是后端 Service 长时间处于 `EXTERNAL-IP=<pending>` 状态。虽然 Deployment 和 Pod 已经正常运行，但外部访问入口始终无法生成，导致后端接口无法通过公网访问。

通过分析发现，该问题并非应用本身故障，而是 LoadBalancer 类型 Service 尚未成功绑定华为云 ELB 资源。为解决该问题，在华为云控制台中创建共享型弹性负载均衡器，并通过 Service 注解指定 ELB 资源，随后 Service 成功获得公网 IP 地址，实现了后端接口的公网访问。

实验过程中还遇到了 kubectl 无法连接集群的问题。最初获得的 kubeconfig 文件使用的是 CloudShell 内网访问模式，其 API Server 地址为私有网络地址，仅能在特定网络环境下访问。当在独立 ECS 上使用该配置文件时，出现 API Server 连接超时错误。经过分析后确认问题来源于网络访问路径而非集群本身故障，后续通过重新获取

正确的集群访问配置完成验证。

此外，在 Service、ConfigMap 和 Secret 等资源部署过程中，也曾出现资源名称拼写错误、命名空间不一致等问题。由于 Kubernetes 对资源名称高度敏感，细微的拼写差异都会导致应用启动失败。通过使用 `kubectl describe` 和 `kubectl logs` 查看详细信息，逐步定位问题并完成修正。

5.1.3 PVC 与 HPA 验证问题

在 Redis 持久化实验中，PVC 创建后曾出现无法立即绑定存储资源的情况。经过检查发现，PVC 绑定过程依赖底层存储插件和 StorageClass 配置，当存储资源尚未完成分配时，PVC 会短暂处于 Pending 状态。等待存储卷创建完成后，PVC 成功进入 Bound 状态，后续 Redis Pod 能够正常挂载持久化卷。

在持久化验证过程中，还需要通过删除 Pod 并观察自动重建后的数据是否保留来证明 PVC 生效。实验初期曾担心 Pod 删除后数据丢失，但经过实际验证，重新创建的 Redis Pod 仍然能够读取此前写入的数据，证明持久化存储配置正确。

在 HPA 验证阶段，由于弹性伸缩依赖 metrics-server 提供资源指标，因此必须首先确认集群能够正常采集 CPU 和内存使用率。实际测试过程中发现，指标更新存在一定延迟，导致副本数不会立即发生变化。经过持续压测和观察后，最终确认 HPA 能够根据负载变化自动调整副本数量。该现象也表明 Kubernetes 的自动伸缩机制具有一定稳定窗口，以避免因短时间波动导致频繁扩缩容。

5.2 第二部分问题记录

5.2.1 Spark 作业提交问题

在 Spark Operator 实验过程中，首先遇到的问题是镜像拉取失败。由于 Spark Operator 默认引用的公共镜像仓库访问速度较慢，导致 Driver Pod 长时间处于镜像拉取阶段。为保证实验顺利进行，将所需镜像同步至华为云 SWR，并统一修改 SparkApplication 中的镜像地址，从而提高镜像获取效率。

在提交 WordCount 和数据分析任务时，还出现过 SparkApplication 创建成功但 Driver Pod 无法正常运行的问题。经过排查发现，部分镜像中缺少分析脚本文件或数据文件。例如镜像中仅包含 Spark 运行环境，而未包含 `analysis.py` 或 `douban_movies.csv`，导致 Driver 启动后立即报错退出。

针对该问题，重新构建 PySpark 镜像，将分析脚本、数据集以及 WordCount 示例程序统一打包到镜像中，并通过 Docker 容器验证镜像内部文件是否存在。经过重新推送镜像后，SparkApplication 能够顺利完成运行。

此外，在性能分析实验中，还遇到 Pandas 和 Matplotlib 依赖缺失的问题。由于基础 PySpark 镜像未预装相关 Python 数据分析库，导致性能对比代码无法正常执行。后续通过在 Dockerfile 中安装 Pandas 和 Matplotlib，并重新构建镜像，成功解决依赖

缺失问题。

5.2.2 OBS 或本地数据读取问题

在数据分析实验中，数据集读取路径配置也是一个重要问题。最初提交 Spark 作业时，程序默认读取本地路径中的 CSV 文件，但镜像内部并不存在对应文件，导致 Spark 报出 “Path does not exist” 错误。

经过分析后发现，Spark Driver 运行环境与本地开发环境并不一致，必须保证数据文件位于镜像内部指定位置，或通过对对象存储服务进行远程访问。因此后续选择将豆瓣电影数据集直接打包到镜像中，并统一放置于 `/opt/spark/work` 目录，从而确保 Driver 和 Executor 能够访问同一数据源。

该问题说明，在分布式环境中必须明确数据与程序的部署位置，仅在本地验证成功并不能保证集群环境能够正常运行。

5.2.3 性能测试误差问题

在 Pandas 与 PySpark 性能对比实验中，实验结果并未呈现理想情况下的线性加速现象。理论上 Executor 数量增加后应获得更高的执行效率，但实际测试结果受到任务启动开销、数据规模、序列化开销和 Shuffle 通信等多种因素影响。

特别是在数据规模相对有限的情况下，Spark 作业的启动时间和资源调度时间占据了较大比例，使得并行计算优势无法完全体现。此外，不同运行环境之间的缓存状态、CPU 资源占用情况以及网络负载差异，也会对最终结果产生一定影响。

为降低实验误差，本项目统一采用相同数据集和相同查询逻辑进行比较，并结合 Amdahl 定律对加速比进行分析，而不是单纯以运行时间长短评价并行计算性能。通过理论与实验结果相结合的方式，更能够客观反映分布式计算系统的实际性能特征。

第六章 总结与收获

本课程设计围绕云计算平台搭建与 Spark 分布式数据分析两个方向展开，涵盖了 Docker 容器化、Kubernetes 集群部署、配置管理、持久化存储、弹性伸缩以及 Spark 大数据处理等多个核心内容。通过在华为云真实环境中的实践操作，不仅完成了任务书要求的各项实验内容，也进一步加深了对云原生架构和分布式计算体系的理解。本章结合项目实施过程，对课程设计中的主要收获、技术体会以及后续改进方向进行总结。

6.1 云计算平台搭建总结

6.1.1 容器化与 Kubernetes 部署理解

在传统软件部署模式下，应用通常依赖固定服务器环境，不同系统之间容易出现依赖版本不一致、运行环境差异等问题。而在本课程设计中，通过 Docker 技术将应用及其运行环境封装为标准镜像，实现了“一次构建，到处运行”的部署模式。

在容器化实践过程中，进一步理解了镜像、容器、镜像仓库以及容器编排之间的关系。Dockerfile 负责定义应用运行环境，Docker Compose 用于本地多容器协同验证，而 SWR 则承担镜像集中存储与分发功能。通过这一过程，认识到容器技术不仅能够简化部署流程，还能够显著提高应用迁移与扩展能力。

在 Kubernetes 部署阶段，重点学习了 Deployment、Pod 和 Service 等核心资源对象的作用。其中 Deployment 负责应用副本管理与滚动更新，Pod 是 Kubernetes 调度的最小运行单元，而 Service 则为 Pod 提供统一访问入口和服务发现能力。通过实际部署后端、前端和 Redis 服务，进一步理解了 Kubernetes 如何实现应用生命周期管理以及故障自动恢复机制。

此外，通过对 ELB、LoadBalancer Service 和公网访问流程的实践，也更加清晰地认识到云平台网络架构中负载均衡、服务暴露以及流量转发的实现方式。相比本地实验环境，真实云平台更能够体现现代云原生应用的部署模式和运行机制。

6.1.2 配置分离、持久化与弹性伸缩理解

在项目实施过程中，ConfigMap、Secret、PVC 和 HPA 等 Kubernetes 高级资源的使用，使配置管理和运维能力得到了进一步提升。

首先，ConfigMap 与 Secret 的应用使系统配置与应用代码实现解耦。Redis 地址、端口等非敏感配置通过 ConfigMap 管理，而密码等敏感信息则通过 Secret 存储。这种方式避免了将环境配置硬编码到程序中，提高了系统的可维护性和安全性。

其次，通过 Redis 持久化实验，对 PVC 和持久化存储机制有了更加深入的理解。在实验中，即使删除 Redis Pod，数据仍能够在新创建的 Pod 中继续访问，说明 PVC 已经成功将数据与容器生命周期分离。该过程体现了云原生架构中“计算资源可销毁、数据资源可保留”的设计思想。

最后，通过 HPA 自动伸缩实验，进一步认识到云平台资源动态调度的重要性。当系统负载升高时，HPA 能够自动增加应用副本，提高服务处理能力；当负载降低时，又能够自动减少副本数量，从而节约资源成本。这种弹性伸缩能力是云计算区别于传统服务器部署的重要优势之一，也是实现资源高效利用的重要手段。

6.2 Spark 大数据分析总结

6.2.1 分布式数据处理流程理解

在 Spark 大数据分析部分，项目以豆瓣电影数据集为对象，完成了数据清洗、统计分析以及性能测试等任务。通过实际编写 PySpark 程序，对 Spark 的分布式执行机制有了更加直观的认识。

在 Spark 架构中，Driver 负责作业解析、任务调度和执行控制，而 Executor 负责实际的数据计算。用户提交的程序首先由 Driver 转换为执行计划，再将任务分发给多个 Executor 并行执行，最终汇总计算结果。与传统单机程序相比，Spark 能够充分利用集群资源处理更大规模的数据集。

在数据清洗过程中，学习了 Spark DataFrame 的基本使用方法，包括数据加载、Schema 推断、缺失值处理以及统计分析等操作。通过 DataFrame API，可以方便地完成数据转换和预处理任务，并且能够自动优化执行计划，提高运行效率。

在 Spark SQL 部分，进一步掌握了 GROUP BY 聚合分析、Top-N 排序分析、时间趋势分析以及窗口函数分析等常见数据分析方法。与传统数据库 SQL 不同，Spark SQL 可以直接作用于分布式数据集，从而兼顾 SQL 语言的易用性和分布式计算的高性能特点。

通过 Spark Operator 在 Kubernetes 环境中运行 SparkApplication，也进一步理解了 Spark 与云原生平台结合的实现方式。相比传统独立集群部署模式，Spark on Kubernetes 能够利用 Kubernetes 提供的资源调度和弹性管理能力，实现更加灵活的分布式计算环境。

6.2.2 性能对比与 Amdahl 定律理解

在性能分析实验中，通过比较 Pandas、单 Executor PySpark 和双 Executor PySpark 的执行时间，进一步理解了并行计算的性能特征。

实验结果表明，增加 Executor 数量并不一定能够获得完全线性的性能提升。这一现象与 Amdahl 定律的结论一致。根据 Amdahl 定律，程序整体加速比不仅取决于并行部分的执行效率，还受到串行部分比例的限制。当程序中仍存在大量无法并行化的工作时，即使增加计算资源，整体性能提升也会逐渐趋于饱和。

在 Spark 实际运行过程中，任务启动、资源调度、数据序列化、Shuffle 通信以及结果汇总等操作均属于额外开销。当数据规模较小时，这些开销甚至可能超过计算本身的时间消耗。因此，在小规模数据集上，PySpark 的优势并不会十分明显；而在大规模数据场景下，分布式计算的性能优势才会逐渐体现出来。

通过将理论模型与实验结果进行对比分析，不仅验证了 Amdahl 定律的实际意义，也进一步认识到分布式系统设计需要综合考虑计算规模、通信开销和资源利用率等因素，而不能简单依赖增加节点数量提升性能。

6.3 课程设计反思

6.3.1 主要挑战

本课程设计最大的挑战来自于真实云平台环境的复杂性。与课堂实验相比，云环境涉及网络配置、镜像仓库、负载均衡、存储资源以及 Kubernetes 多种资源对象之间的协同工作，任何一个环节出现配置错误都可能导致整体部署失败。

此外，Spark 分布式分析部分涉及镜像构建、依赖安装、数据集管理以及 Kubernetes 作业调度等多个方面，问题定位过程较为复杂。例如分析脚本缺失、数据文件缺失、镜像缓存未更新以及 Python 依赖库缺失等问题，都需要结合日志信息逐步排查才能发现根本原因。

通过解决这些问题，不仅提高了实际工程排错能力，也更加深刻地理解了云计算平台中各组件之间的协作关系。

6.3.2 改进方向

虽然本项目已经完成了课程设计要求的主要功能，但仍存在进一步完善的空间。

首先，可以引入更加规范的 CI/CD 流水线，实现镜像自动构建、自动测试和自动部署，进一步提升项目交付效率。其次，可以将 Spark 数据源扩展到 OBS 对象存储，从而更接近真实大数据处理场景。再次，可以增加监控与日志系统，例如 Prometheus、Grafana 和 ELK，实现应用运行状态和资源使用情况的实时可视化。

在性能分析方面，也可以采用更大规模的数据集和更多 Executor 配置进行测试，以获得更加具有代表性的性能对比结果，并进一步验证分布式计算模型在不同规模场景下的表现差异。

总体而言，本课程设计不仅完成了课程要求的实验任务，更重要的是通过真实云平台实践，将课堂中学习的云计算、容器技术、Kubernetes 和分布式计算知识进行了系统整合与应用，为后续学习和工程实践奠定了良好基础。

第一章 附录：核心配置与代码

A.1 Docker 相关文件

A.1.1 后端 Dockerfile

A.1.2 前端 Dockerfile

A.1.3 docker-compose.yml

A.2 Kubernetes YAML 文件

A.2.1 Deployment 与 Service 配置

A.2.2 ConfigMap 与 Secret 配置

A.2.3 PVC 与 HPA 配置

A.3 Spark 作业代码

A.3.1 wordcount.py

A.3.2 analysis.py

A.3.3 性能测试脚本

第二章 附录：实验截图索引

B.1 第一部分截图索引

B.1.1 任务 1 至任务 3 截图

B.1.2 任务 4 至任务 6 截图

B.2 第二部分截图索引

B.2.1 A-0 与 A-1 截图

B.2.2 A-2 与 A-3 截图